Monte Carlo Tree Search and AlphaGo

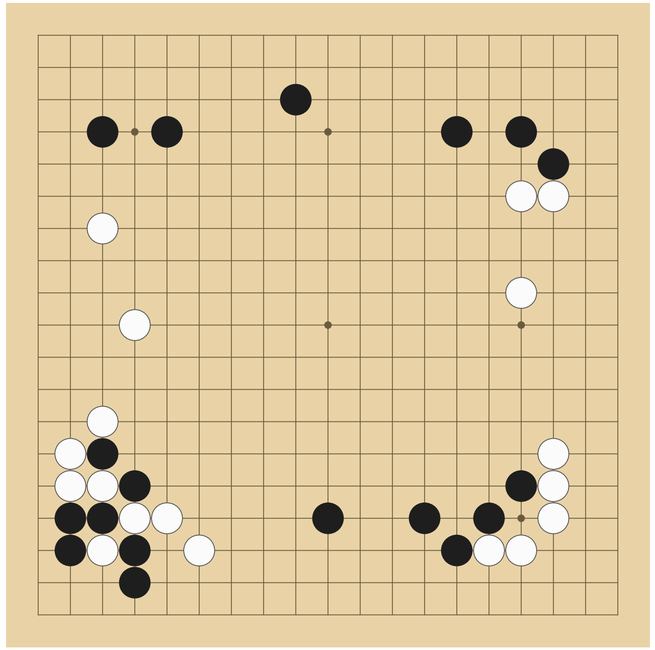
Big leagues 😎

Dhruman Gupta & Rushil Gupta

Why Go?

Simple rules, absurd numbers

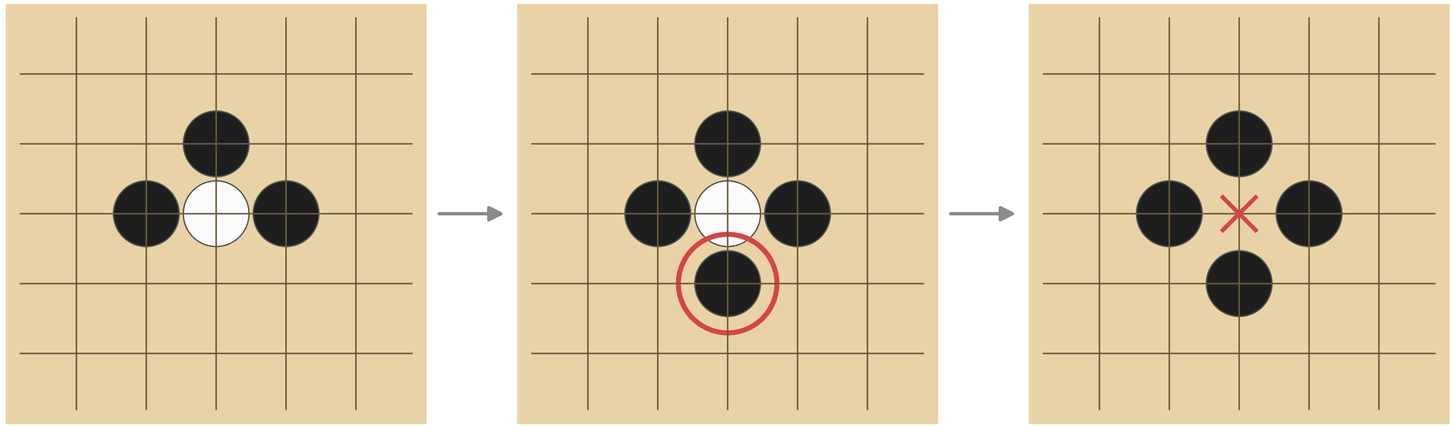
The Game of Go



Two players take turns placing stones on a 19 × 19 board

Black moves first. Stones never move once placed.

The Game of Go



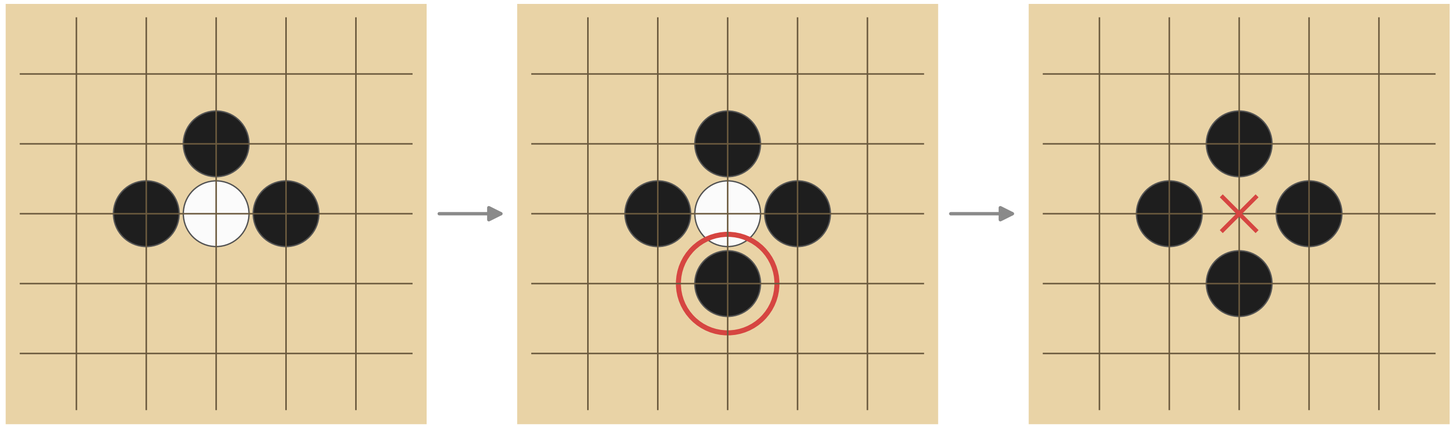
one liberty left

black fills it

white is captured

A group with no empty neighbours is captured

The Game of Go



one liberty left

black fills it

white is captured

A group with no empty neighbours is captured

Whoever controls more of the board at the end wins

Go as an RL Problem

State s: the board, and whose turn it is

Go as an RL Problem

State s: the board, and whose turn it is

Action a: place a stone on any empty point, or pass

Go as an RL Problem

State s: the board, and whose turn it is

Action a: place a stone on any empty point, or pass

Reward: $+1$ for a win, -1 for a loss

Only at the very end of the game

Go as an RL Problem

State s: the board, and whose turn it is

Action a: place a stone on any empty point, or pass

Reward: $+1$ for a win, -1 for a loss

Only at the very end of the game

We know the rules exactly...

So can't we just search for the best move?

How Big Is Go?

Every point is empty, black, or white

$$3^{361} \approx 1.7 \times 10^{172}$$

How Big Is Go?

Every point is empty, black, or white

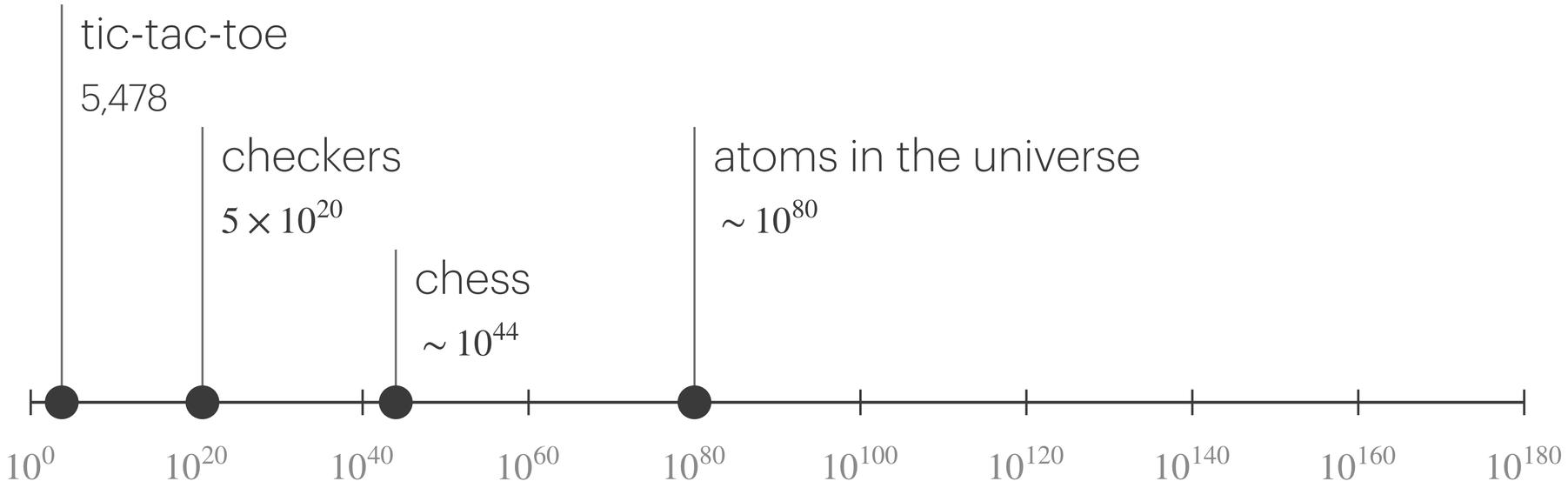
$$3^{361} \approx 1.7 \times 10^{172}$$

About 1.2% of those are legal positions

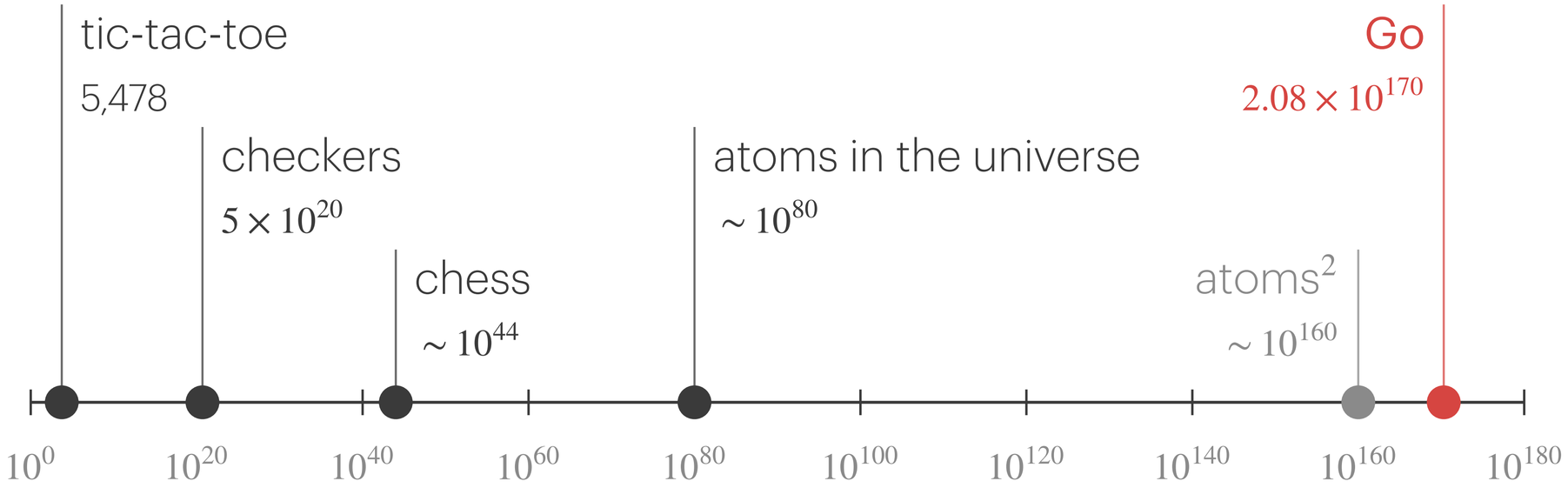
$$2.08 \times 10^{170}$$

Counted exactly by Tromp & Farnebäck, 2016

Is That a Lot?

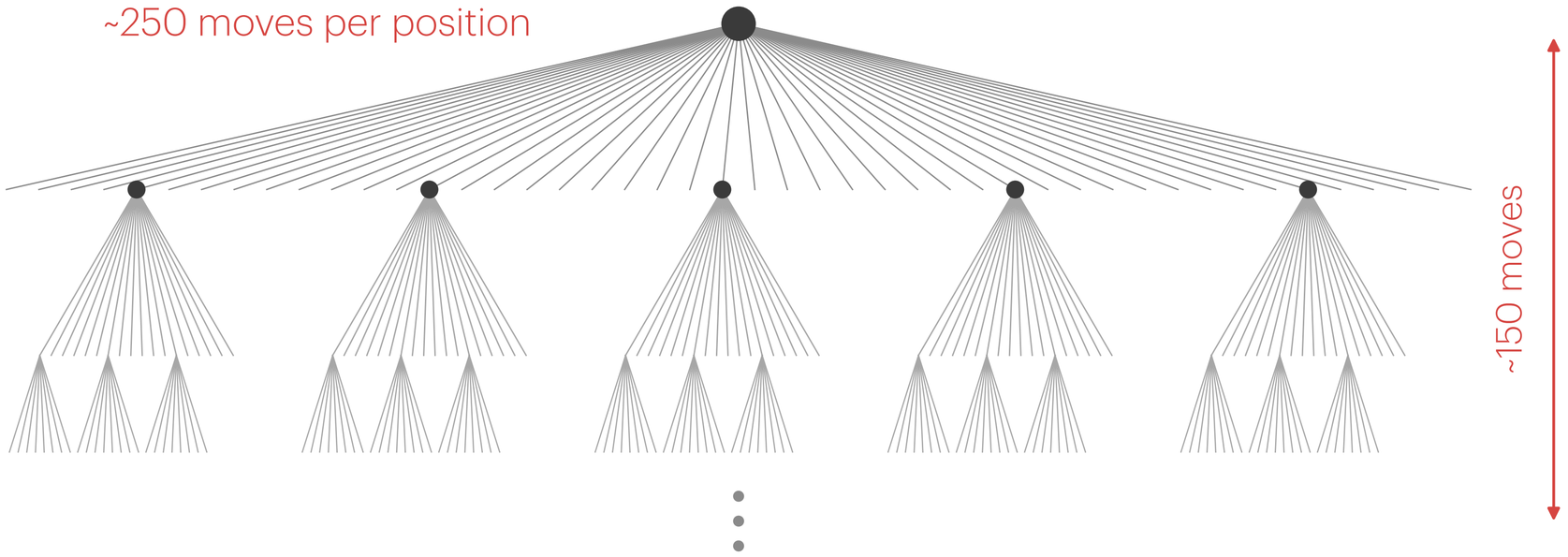

tic-tac-toe
5,478
checkers
5×10^{20}
chess
$\sim 10^{44}$
atoms in the universe
$\sim 10^{80}$
10^{0}
10^{20}
10^{40}
10^{60}
10^{80}
10^{100}
10^{120}
10^{140}
10^{160}
10^{180}

Is That a Lot?

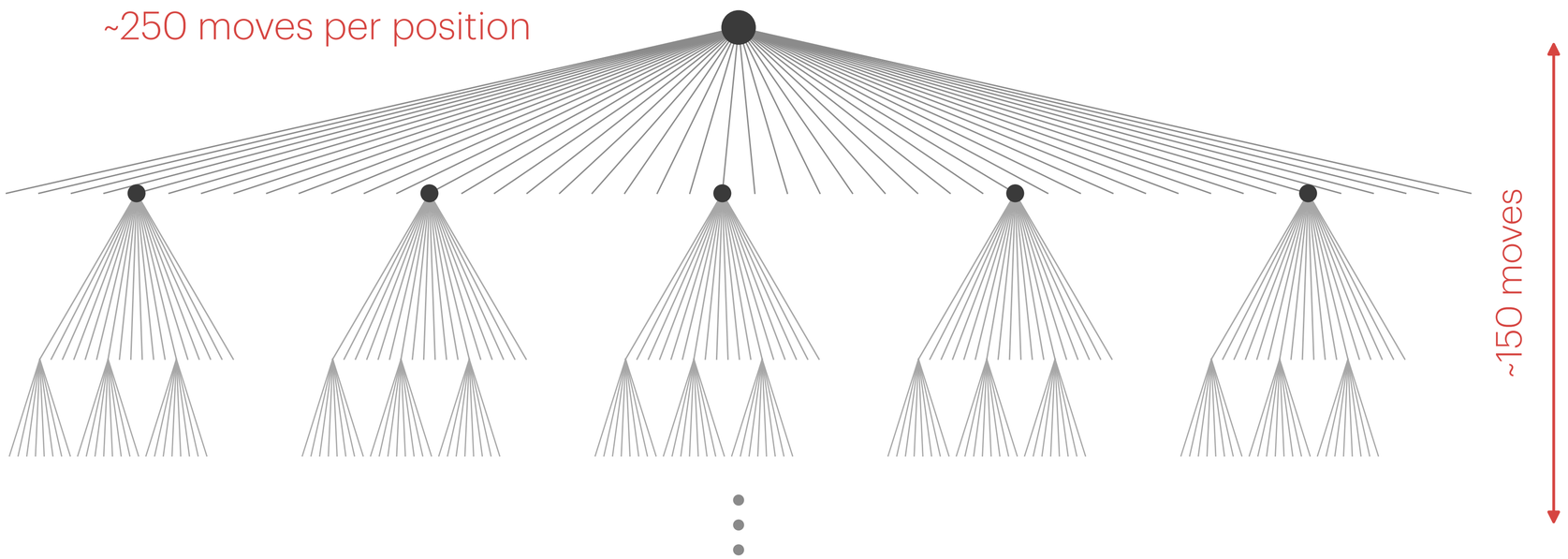


More positions than atoms in the universe, squared

How Big Is the Game Tree?



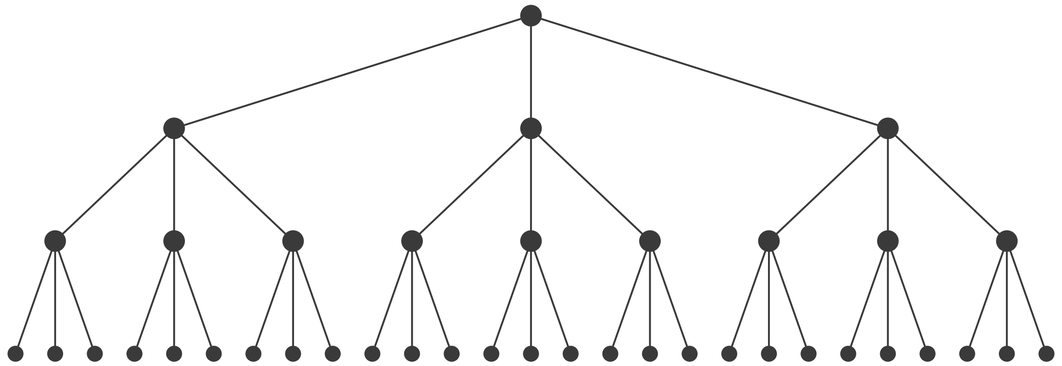
How Big Is the Game Tree?



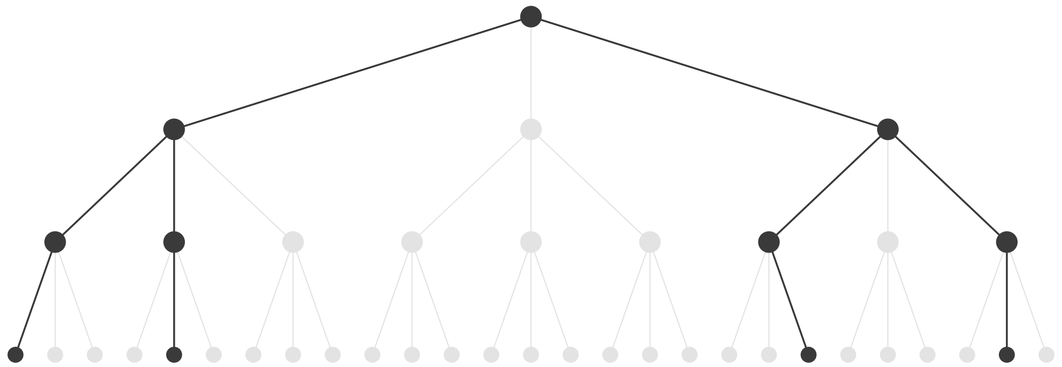
$$250^{150} \approx 10^{360}$$

possible games (chess: ~10^{120})

Searching Cleverly



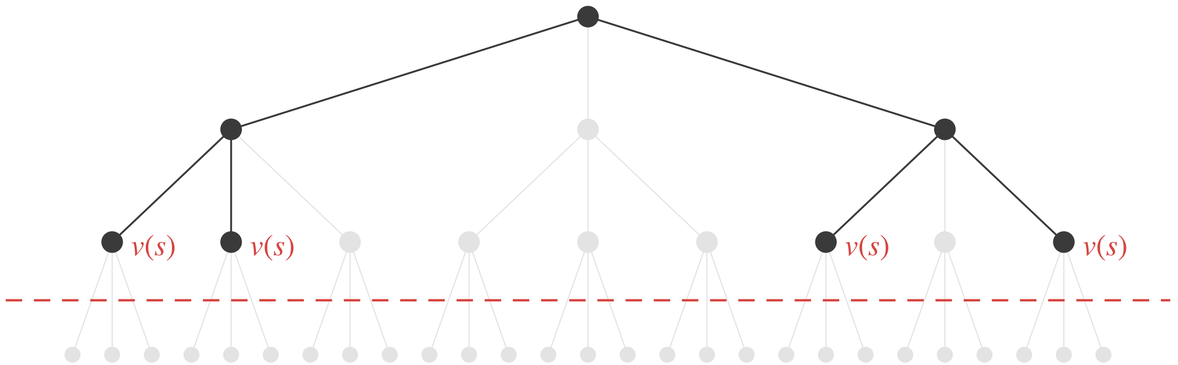
Searching Cleverly



Breadth: only look at promising moves

a policy $p(a \mid s)$

Searching Cleverly



Depth: stop early and estimate who's winning

a value $v(s)$

Roadmap

AlphaGo: what happened

Roadmap

AlphaGo: what happened

Monte Carlo Tree Search

Roadmap

AlphaGo: what happened

Monte Carlo Tree Search

Inside AlphaGo

AlphaGo

Seoul, March 2016

Could a Computer Beat a Champion?

Oct 2015: AlphaGo beats European champion Fan Hui, 5–0

First program to beat a professional on a full board, no handicap

Could a Computer Beat a Champion?

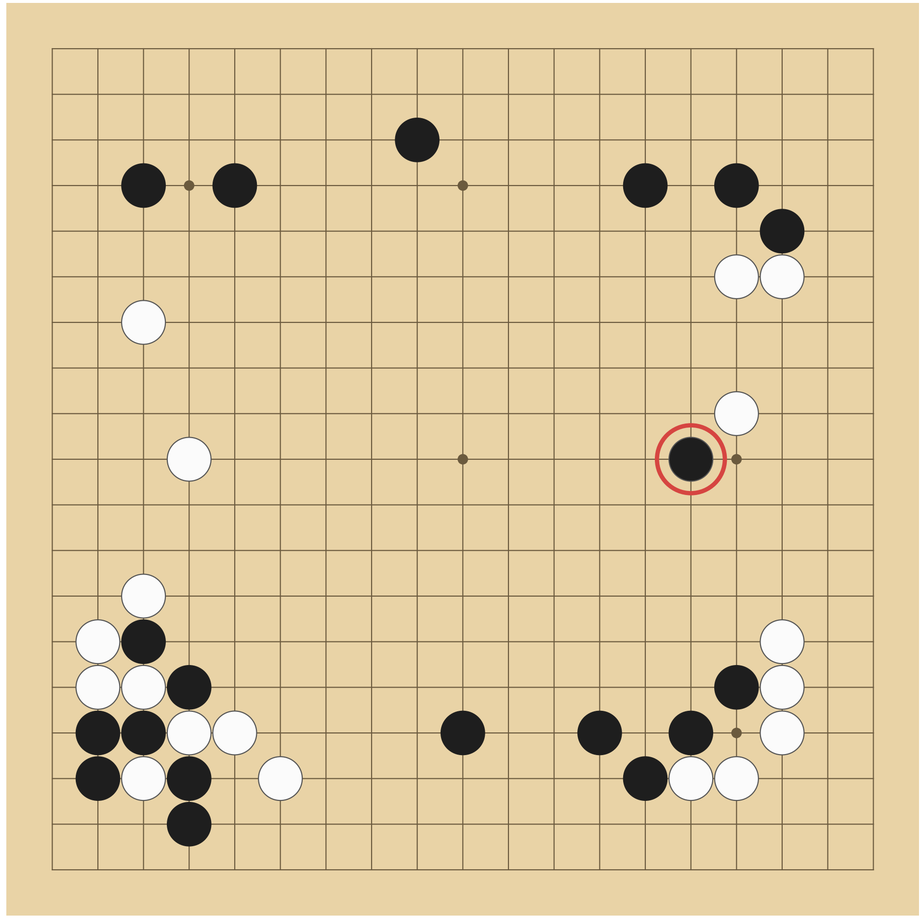
Oct 2015: AlphaGo beats European champion Fan Hui, 5–0

First program to beat a professional on a full board, no handicap

Mar 2016: AlphaGo beats Lee Sedol, 4–1

One of the world's top players

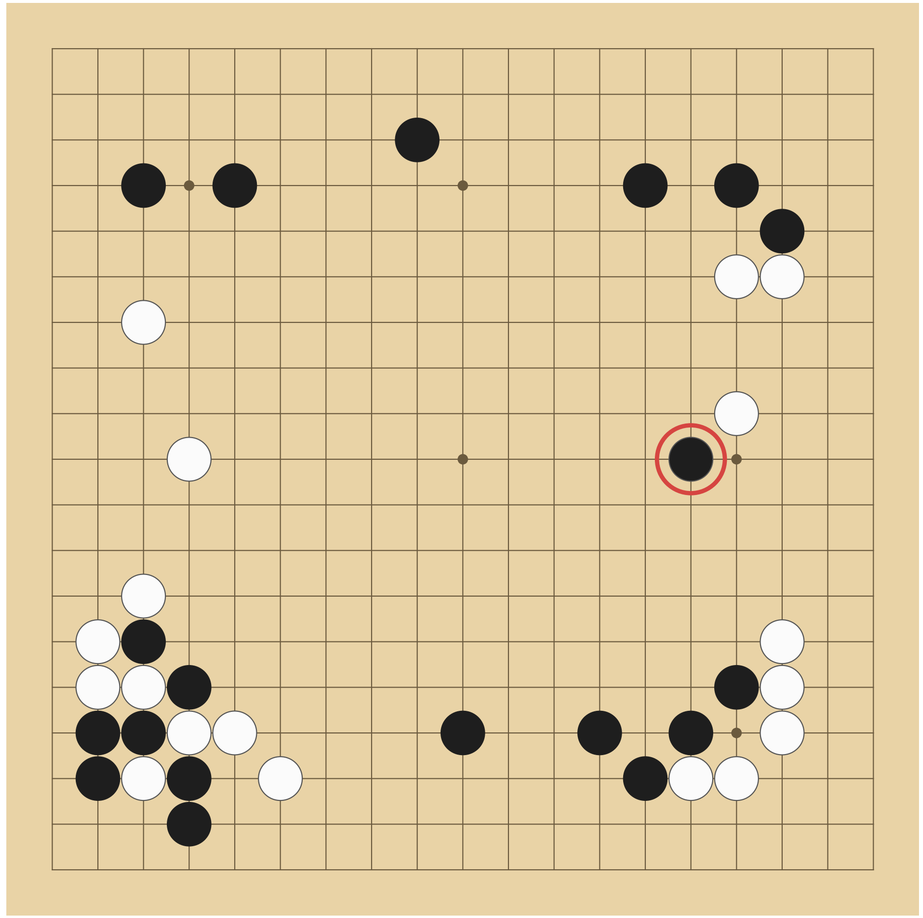
Game 2, Move 37



A fifth-line shoulder hit

AlphaGo (black) at P10

Game 2, Move 37



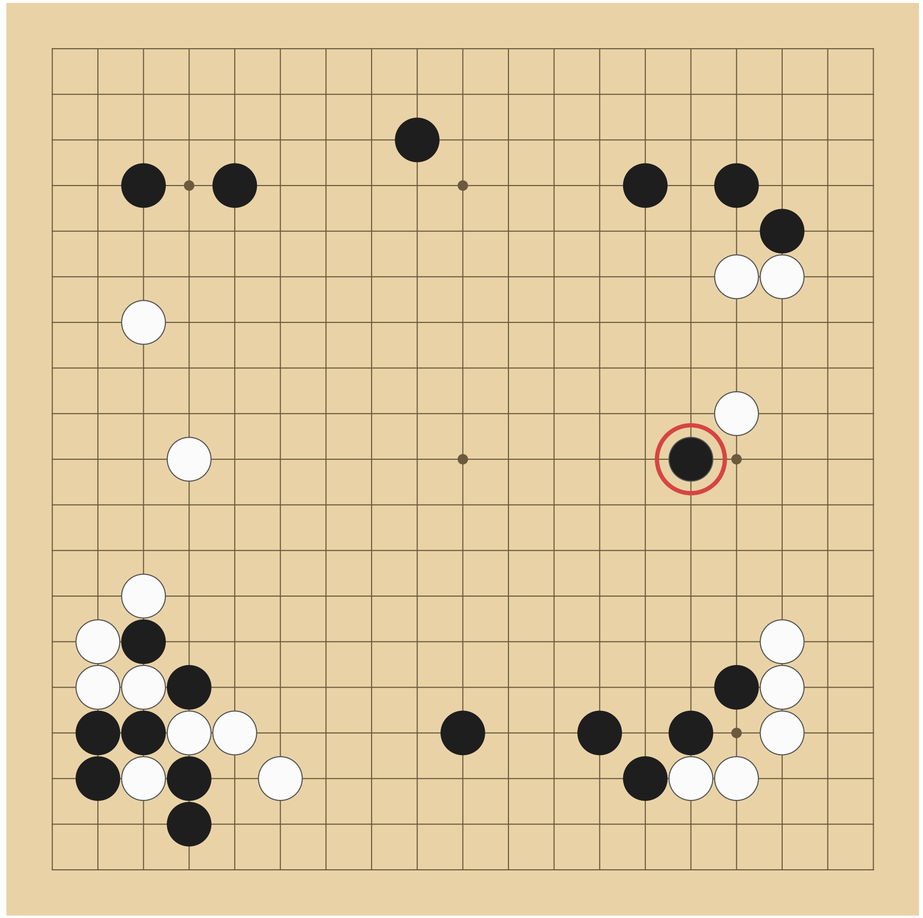
A fifth-line shoulder hit

AlphaGo (black) at P10

Commentators thought it was a mistake

A move no human would play

Game 2, Move 37



A fifth-line shoulder hit

AlphaGo (black) at P10

Commentators thought it was a mistake

A move no human would play

AlphaGo won the game

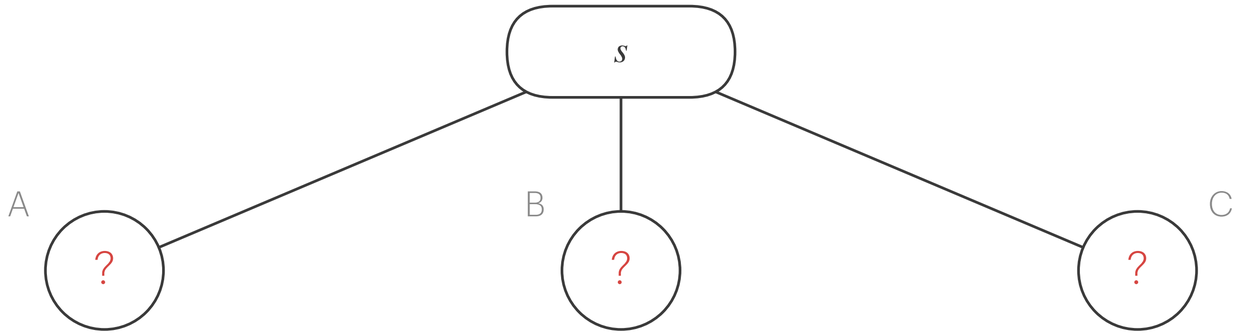
Lee Sedol was now 2–0 down

Monte Carlo Tree Search

When in doubt, play it out

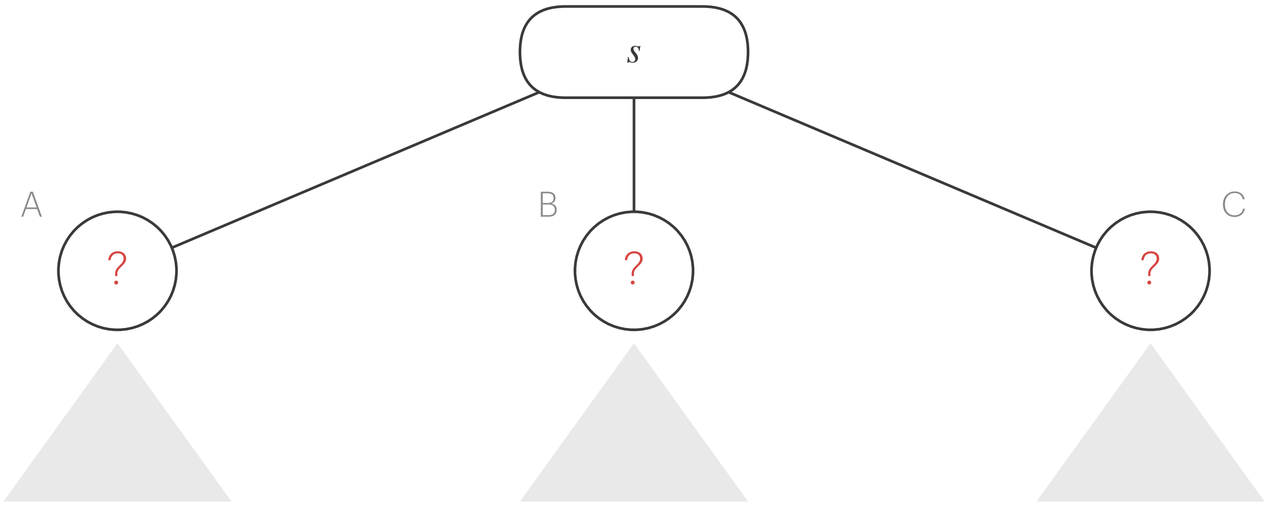
What Is a Move Worth?

To choose a move, we need to know how good each one is



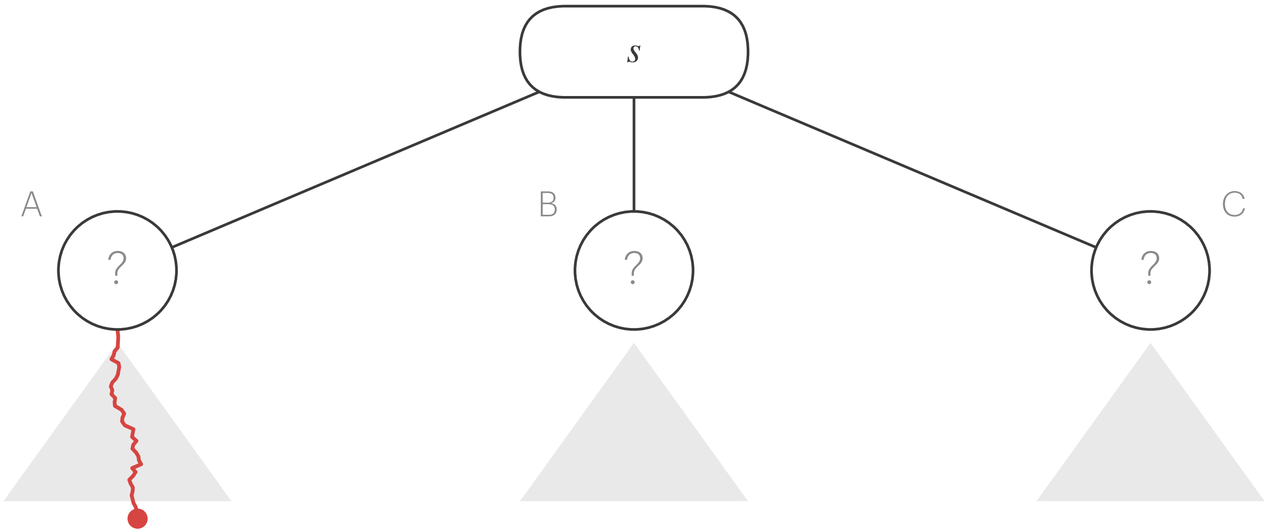
What Is a Move Worth?

We can't search to the end, and nobody can write a good $v(s)$ for Go



Idea: Just Play It Out

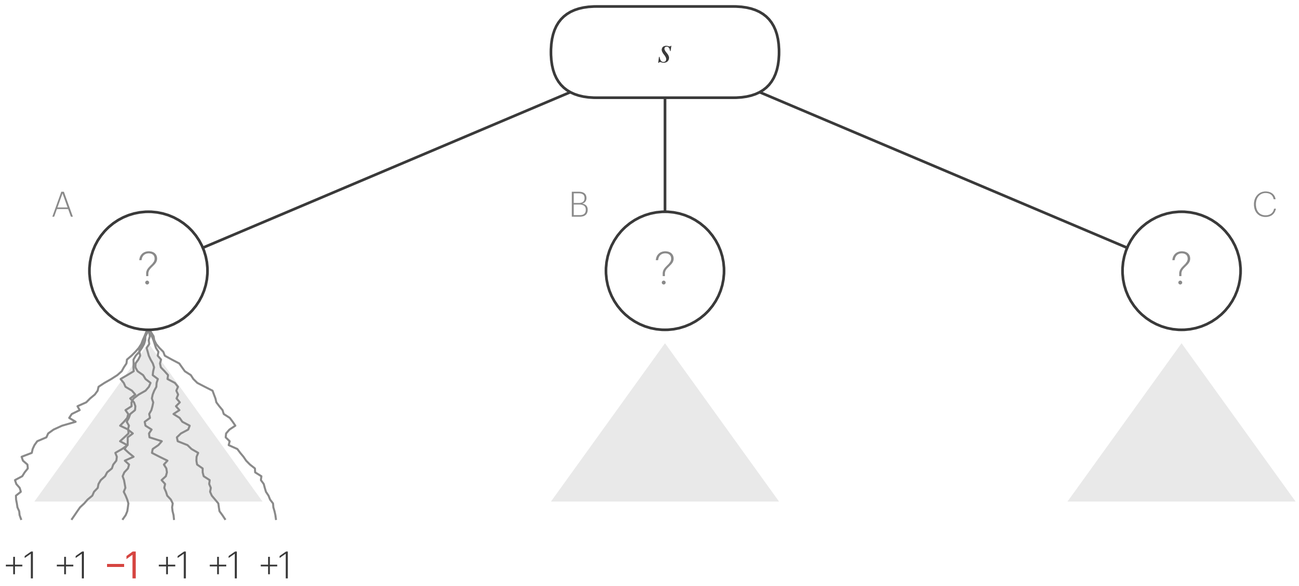
Pick a move, then play random moves until the game ends



Like a Monte Carlo return, but at decision time

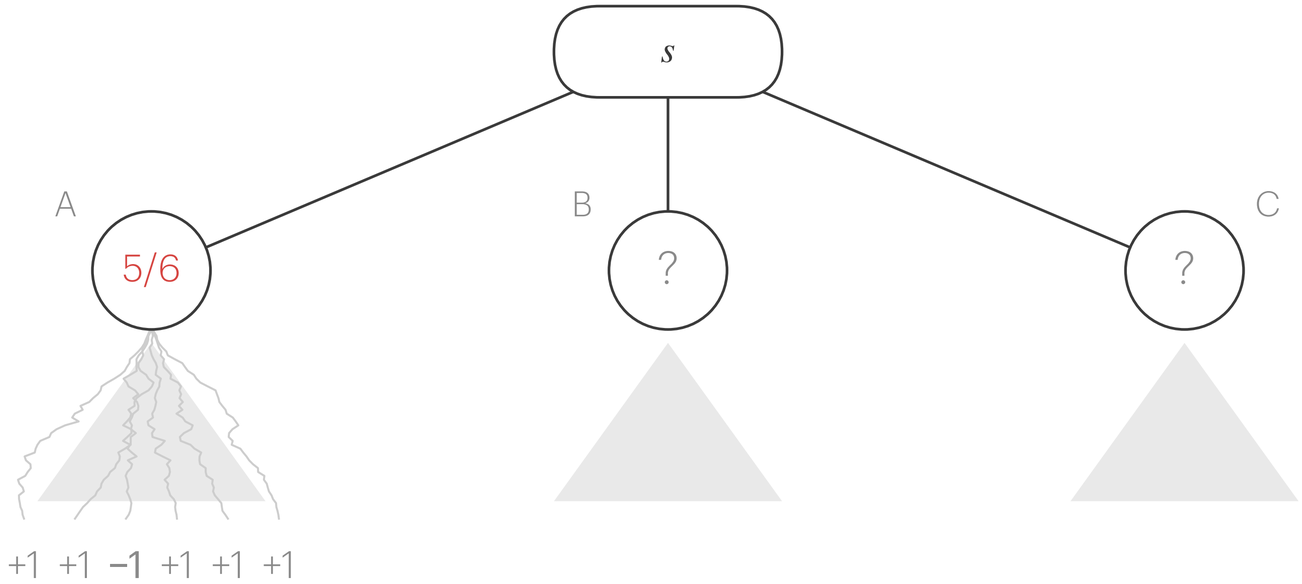
Idea: Just Play It Out

Record who won: z = +1 or −1



Idea: Just Play It Out

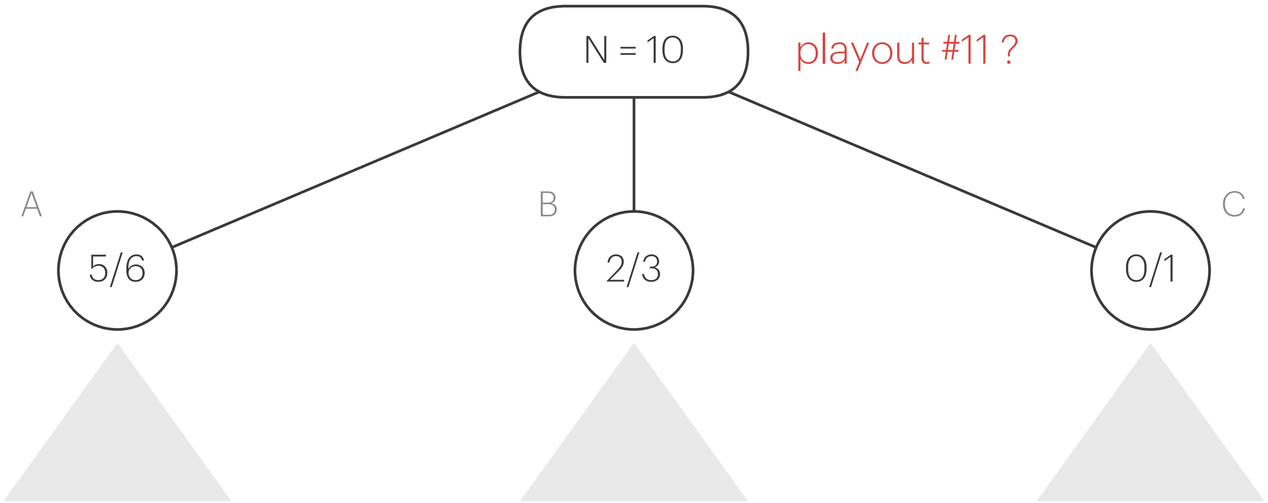
Average the results: a Monte Carlo estimate of the move's value



$$\hat{V}(s) = \frac{1}{n} \sum_{i=1}^{n} z_i$$

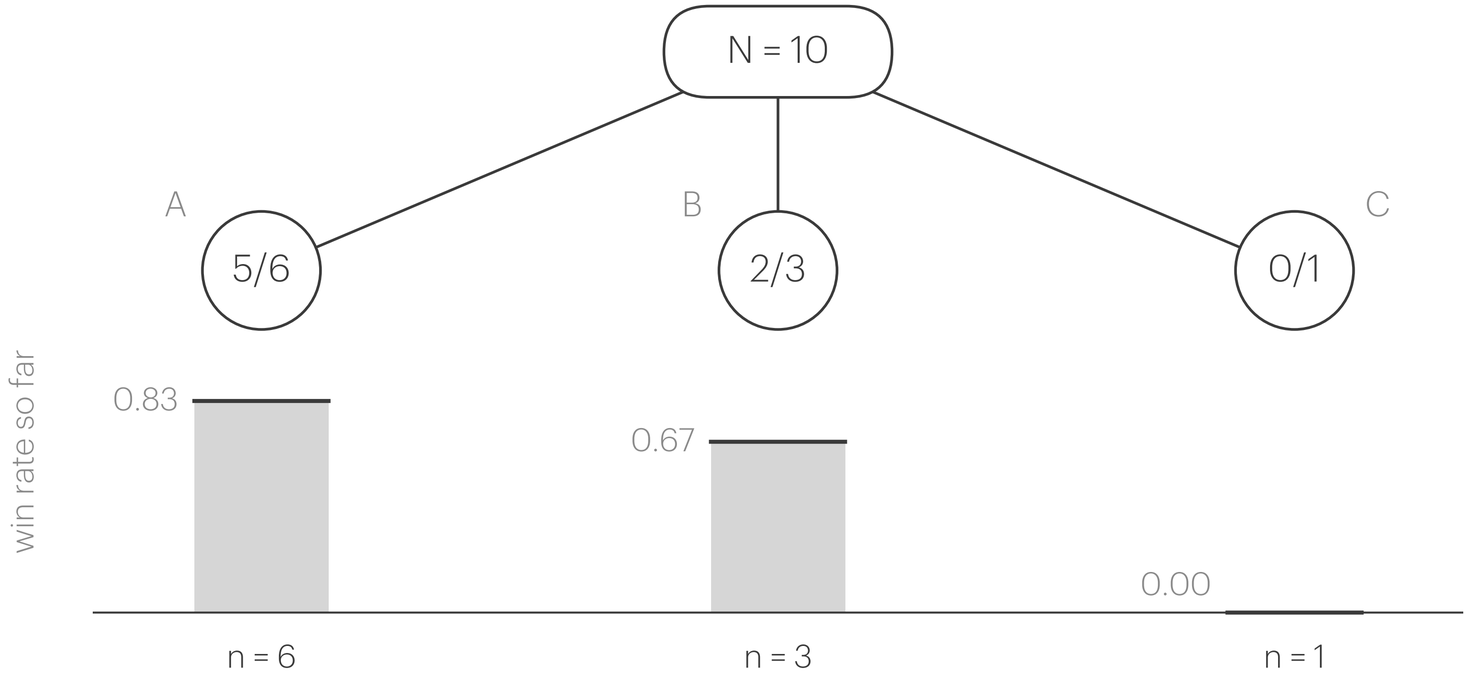
Which Move Gets Playout #11?

10 playouts so far. Which move gets number 11?



Which Move Gets Playout #11?

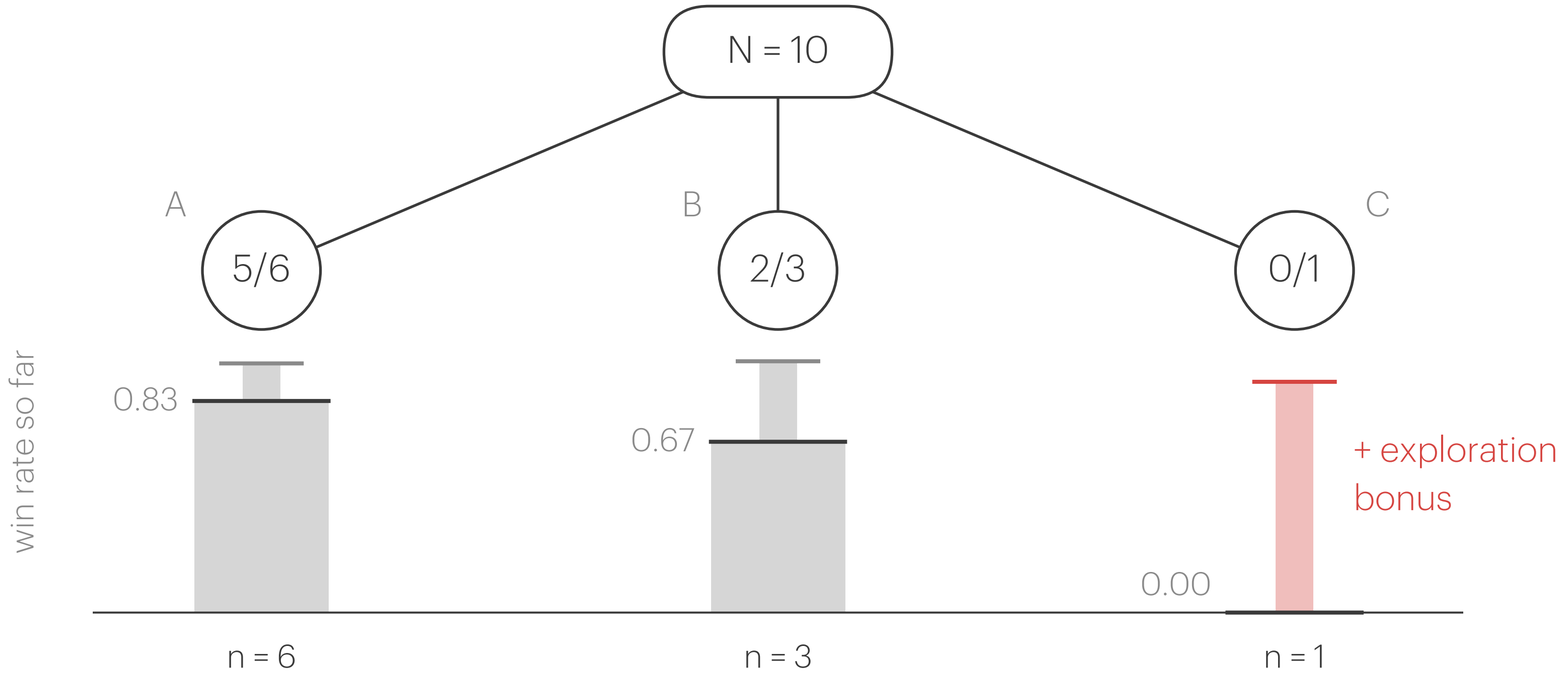
Exploit: A has the best win rate so far



Exploration vs exploitation, again

Which Move Gets Playout #11?

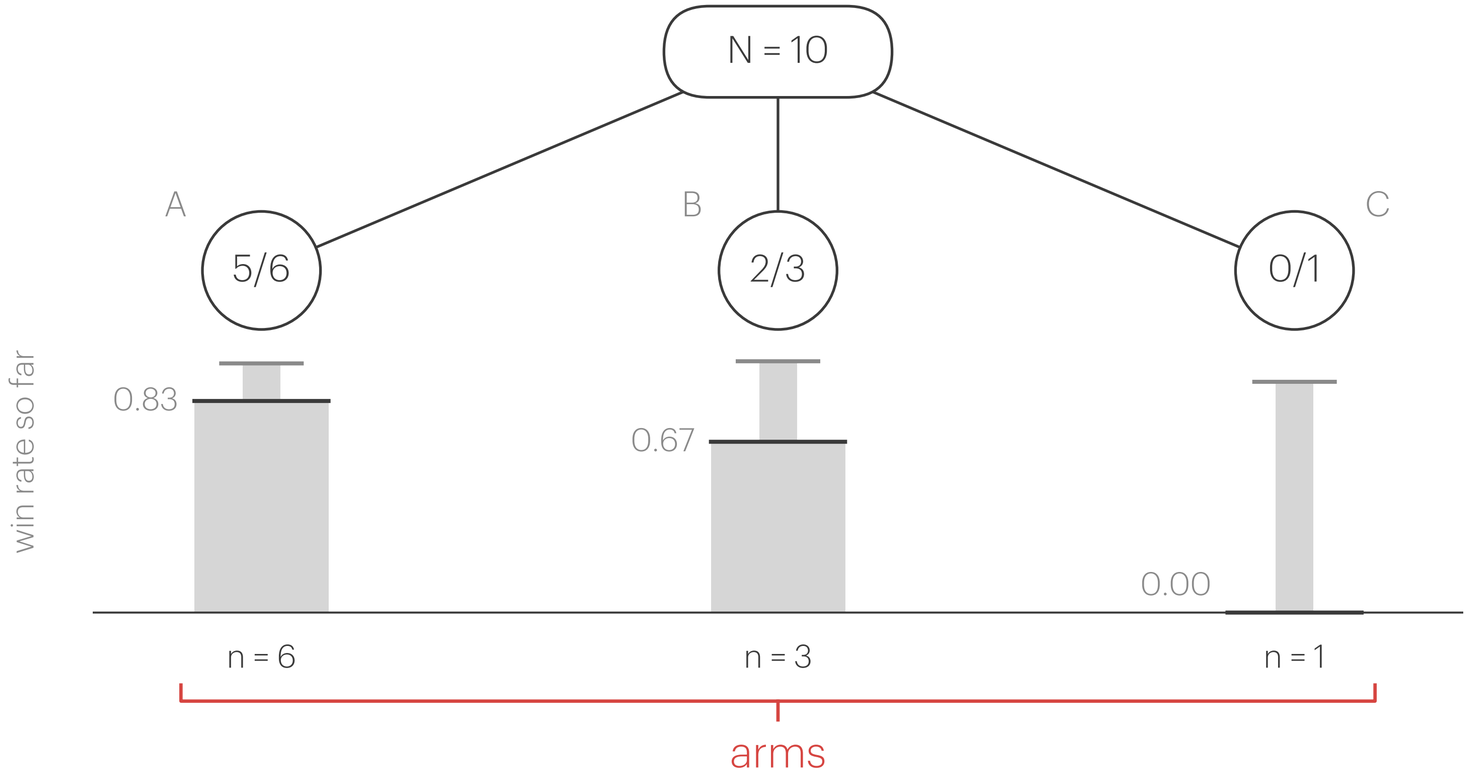
Explore: C was tried once; its real win rate could be anything



Exploration vs exploitation, again

Which Move Gets Playout #11?

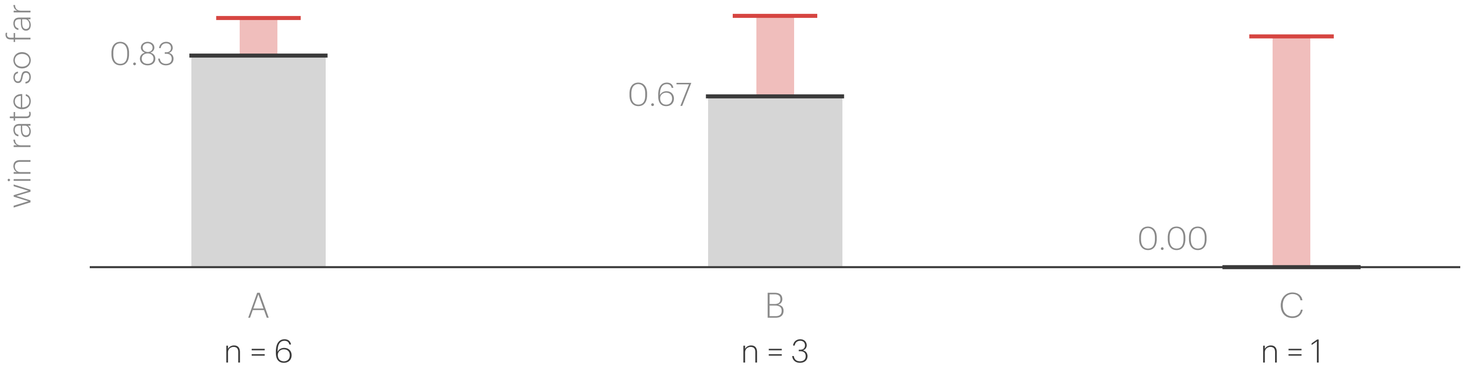
A multi-armed bandit: each move is a slot machine with an unknown win rate



Which Move Gets Playout #11?

Pick the arm with the highest optimistic estimate

c = $\sqrt{2}$ ≈ 1.41 in theory; tuned in practice

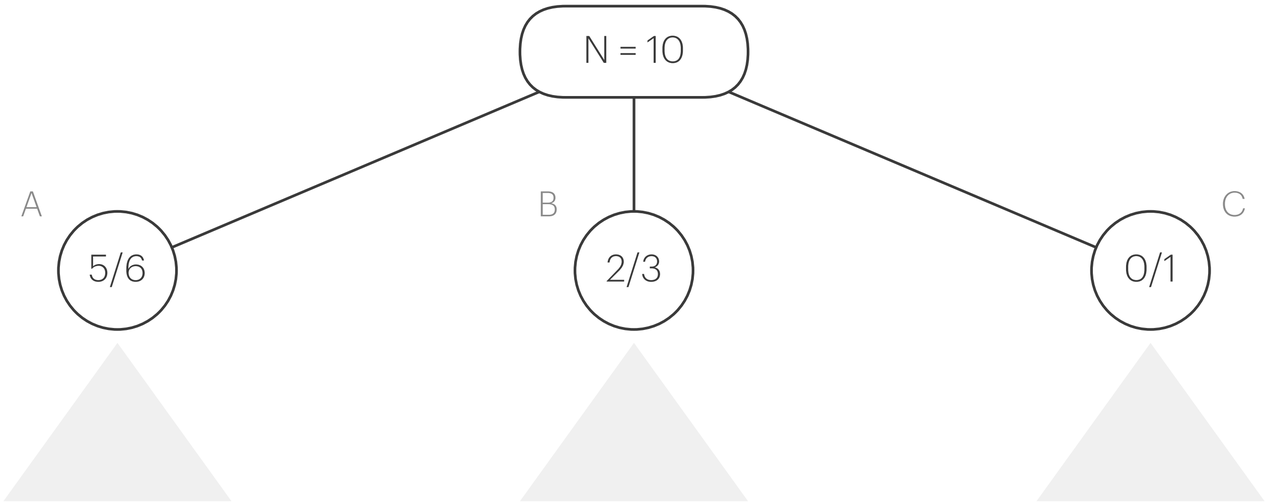


$$\mathrm{UCT}(i) = \underbrace{\frac{W_i}{N_i}}_{\text{exploit}} + \underbrace{c\sqrt{\frac{\ln N}{N_i}}}_{\text{explore}}$$

N: visits to the parent N_i: visits to child i W_i: wins for whoever moved into i

Which Arm Does UCT Pick?

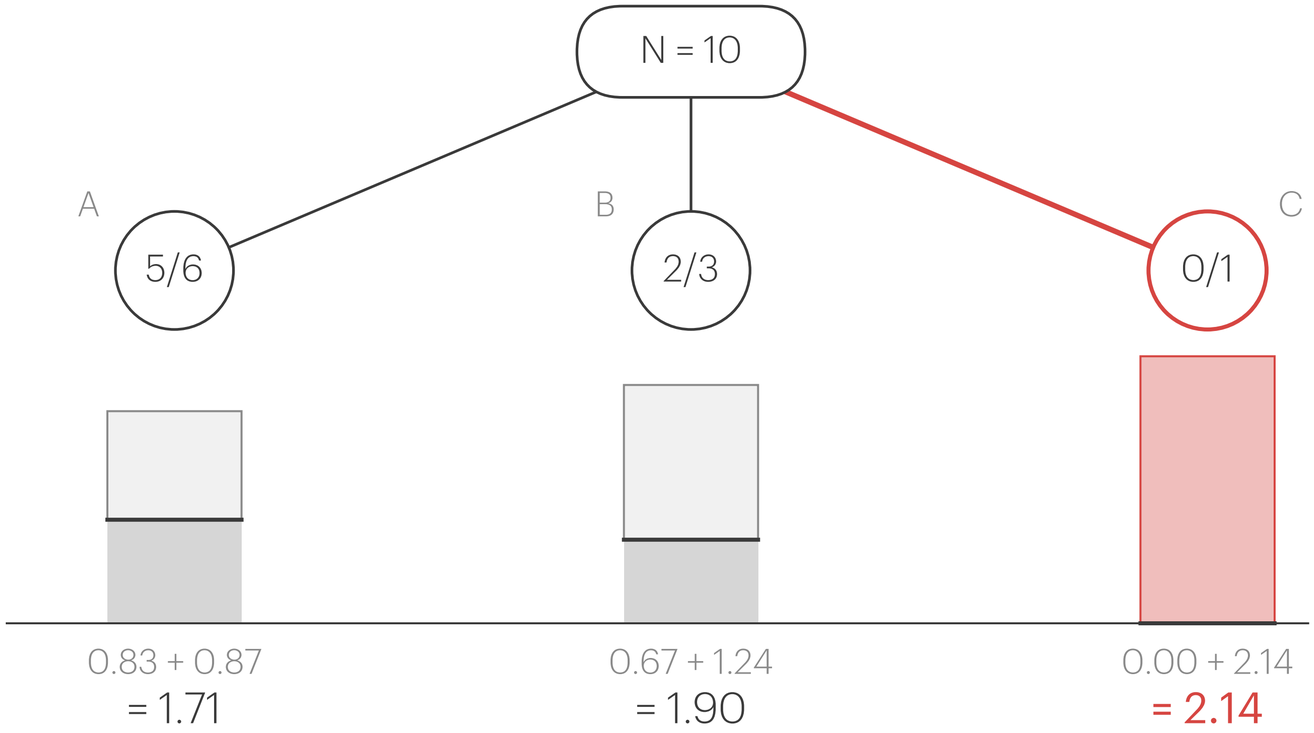
c = 1.41. Which arm does UCT pick?



$$\mathrm{UCT}(i) = \underbrace{\frac{W_i}{N_i}}_{\text{win rate}} + \underbrace{c\sqrt{\frac{\ln N}{N_i}}}_{\text{explore}}$$

Which Arm Does UCT Pick?

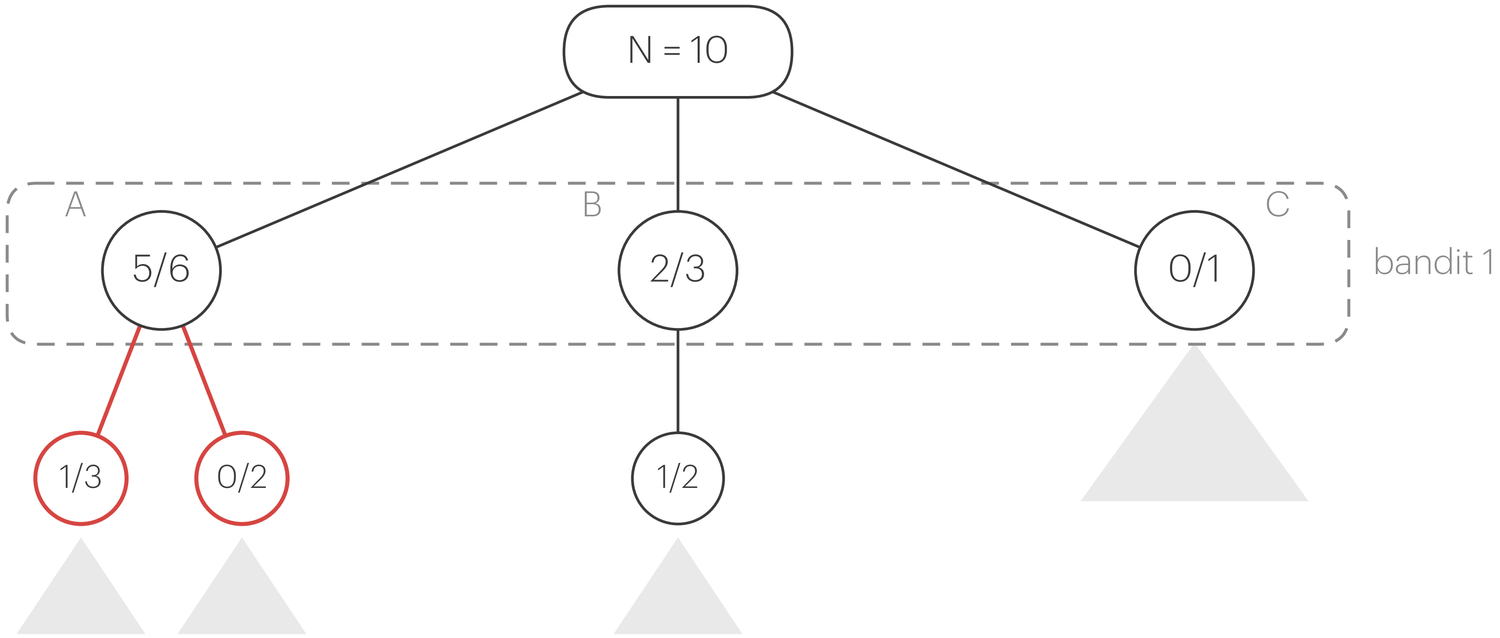
C! Rarely tried beats a high win rate



$$\mathrm{UCT}(i) = \underbrace{\frac{W_i}{N_i}}_{\text{win rate}} + \underbrace{c\sqrt{\frac{\ln N}{N_i}}}_{\text{explore}}$$

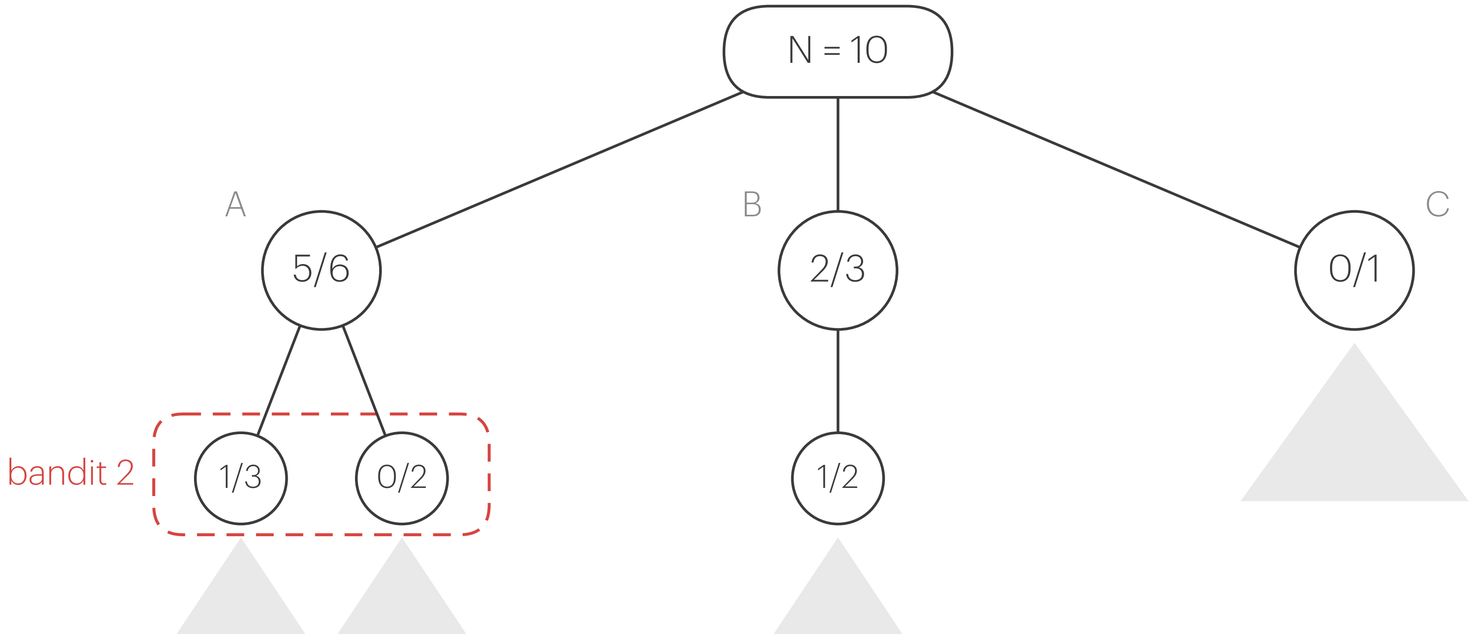
UCT: A Bandit at Every Node

A move leads to the opponent's replies, which are arms too



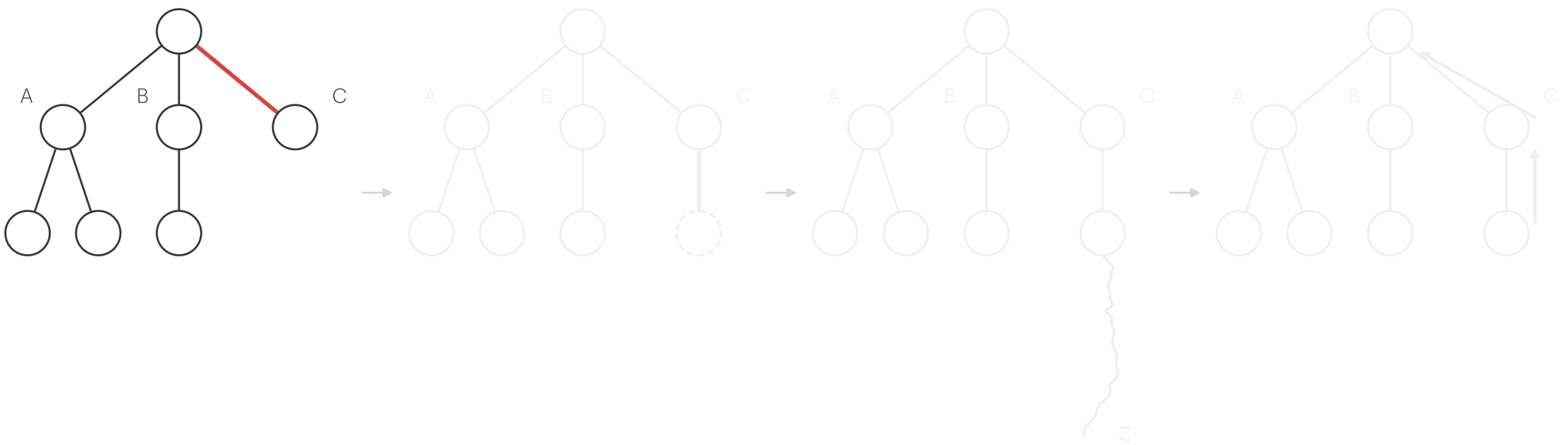
UCT: A Bandit at Every Node

The same UCT rule, one level down



with A's visits as the parent's N

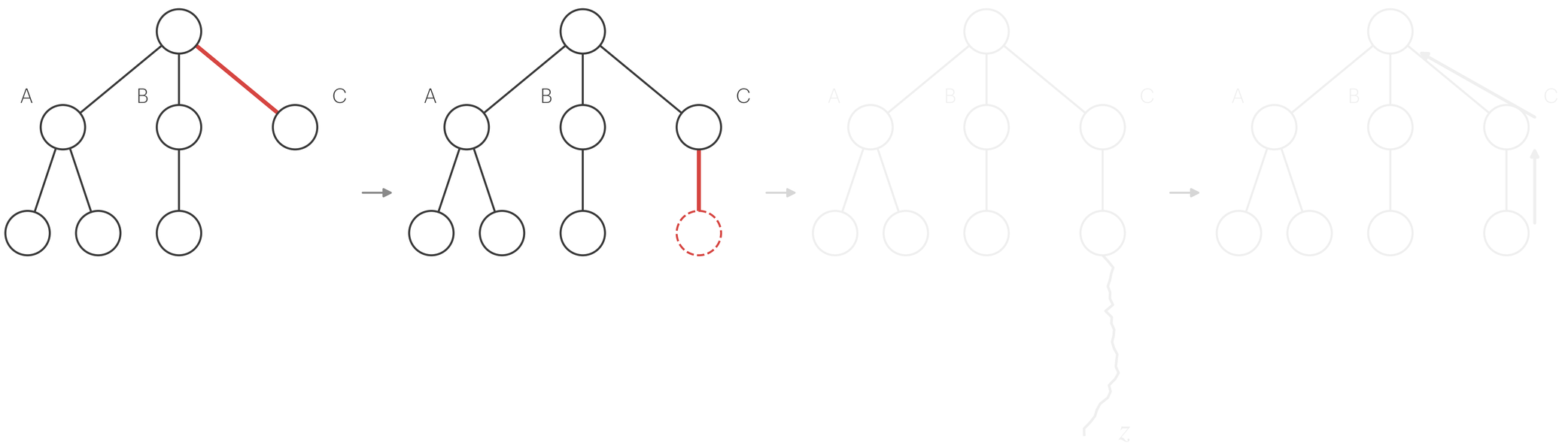
MCTS



Selection Expansion Simulation Backpropagation

Selection: walk down the tree, picking children by UCT

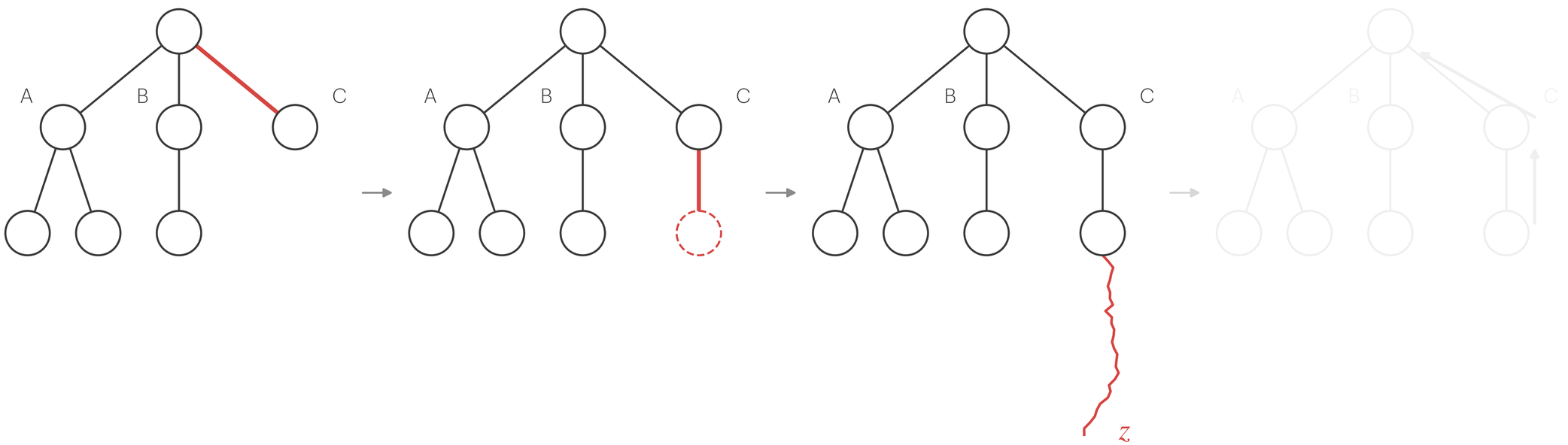
MCTS



Selection Expansion Simulation Backpropagation

Expansion: add one new node to the tree

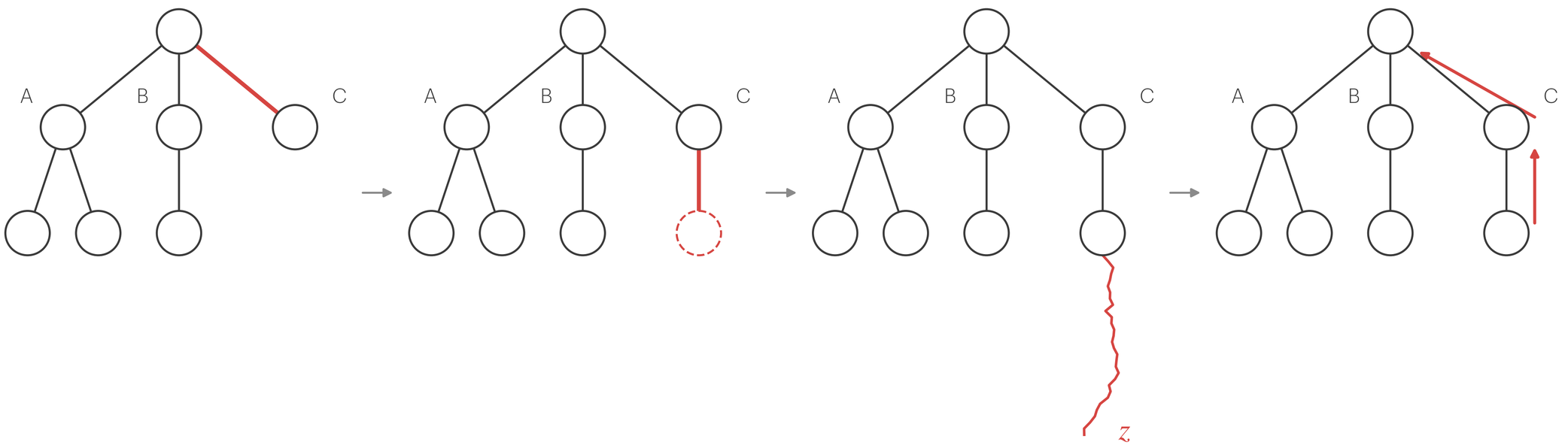
MCTS



Selection Expansion Simulation Backpropagation

Simulation: play randomly from it until the game ends

MCTS



Selection Expansion Simulation Backpropagation

Backpropagation: update win and visit counts on the path

The Algorithm

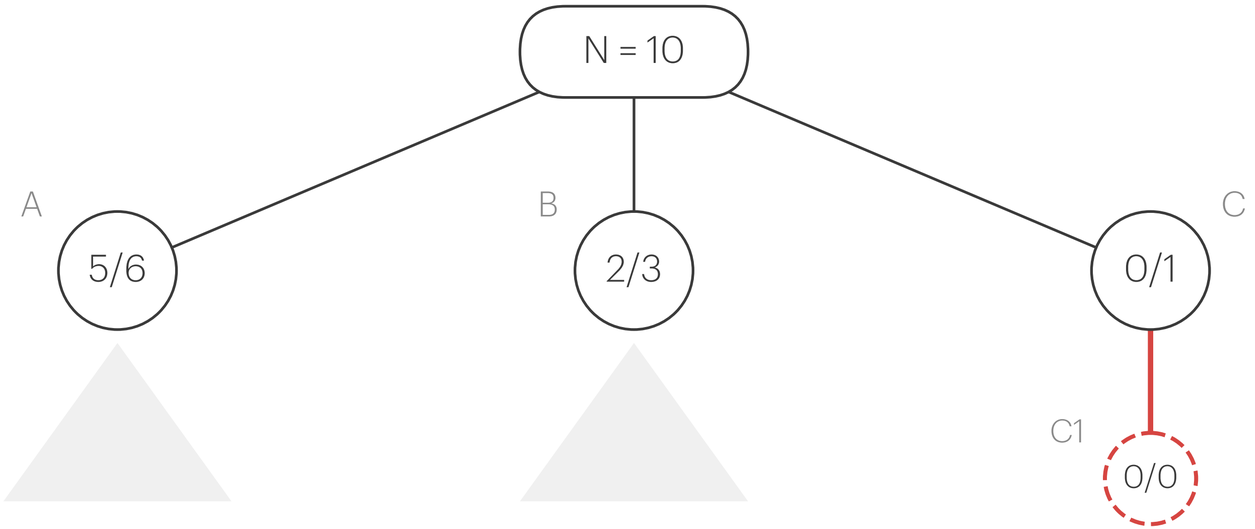
repeat for T simulations:

Selection	$s \leftarrow$ root; while every move from s has been tried, go to the child with the highest UCT score
Expansion	UCT reaches an untried move (its score is ∞): add it to the tree
Simulation	$z \leftarrow$ play randomly from there to the end of the game
Backpropagation	on the path: $N \leftarrow N + 1$, and $W \leftarrow W + 1$ if the player who moved there won

finally, play the move (child of the root) with the most visits

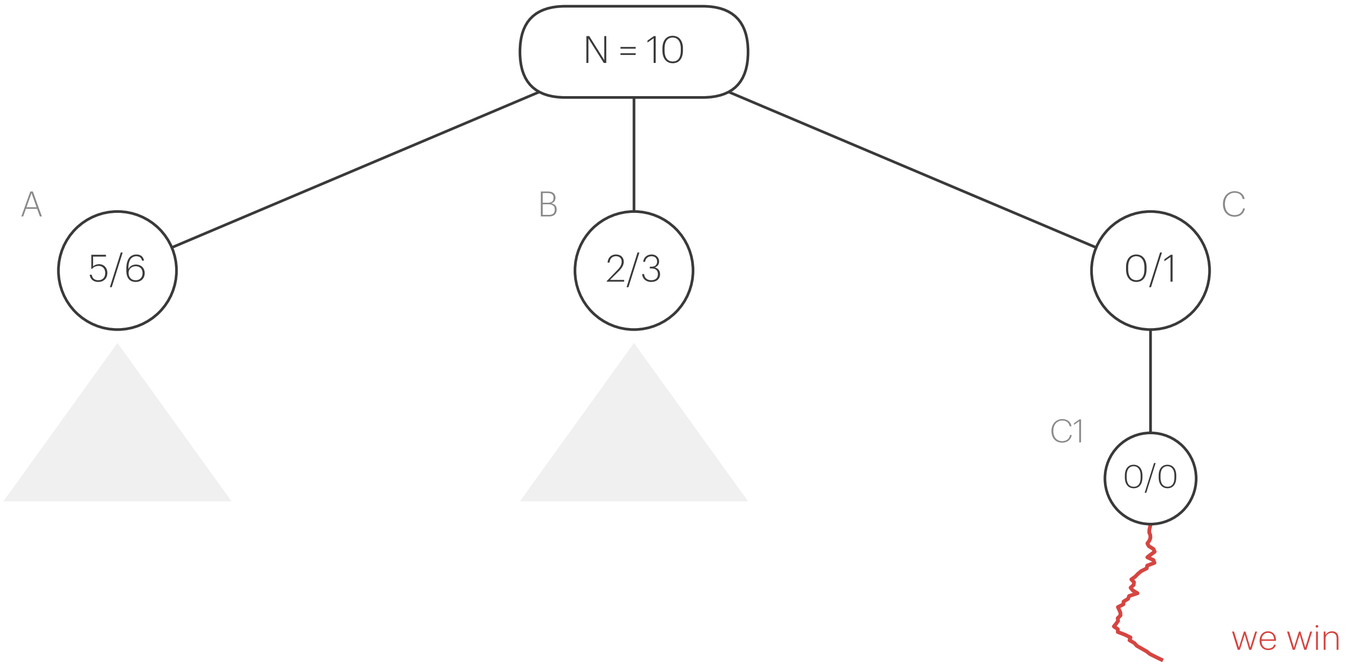
Most visits, not best win rate: visits are robust to lucky streaks

One Simulation, Step by Step



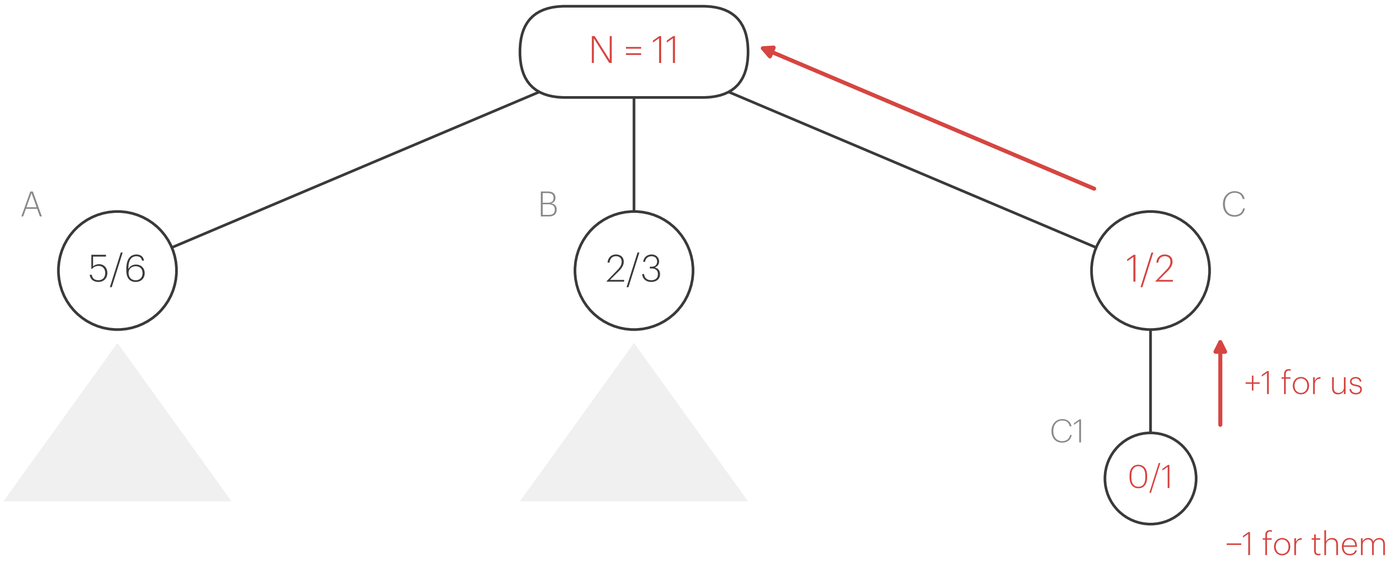
Expand: add a new child C1 under C

One Simulation, Step by Step



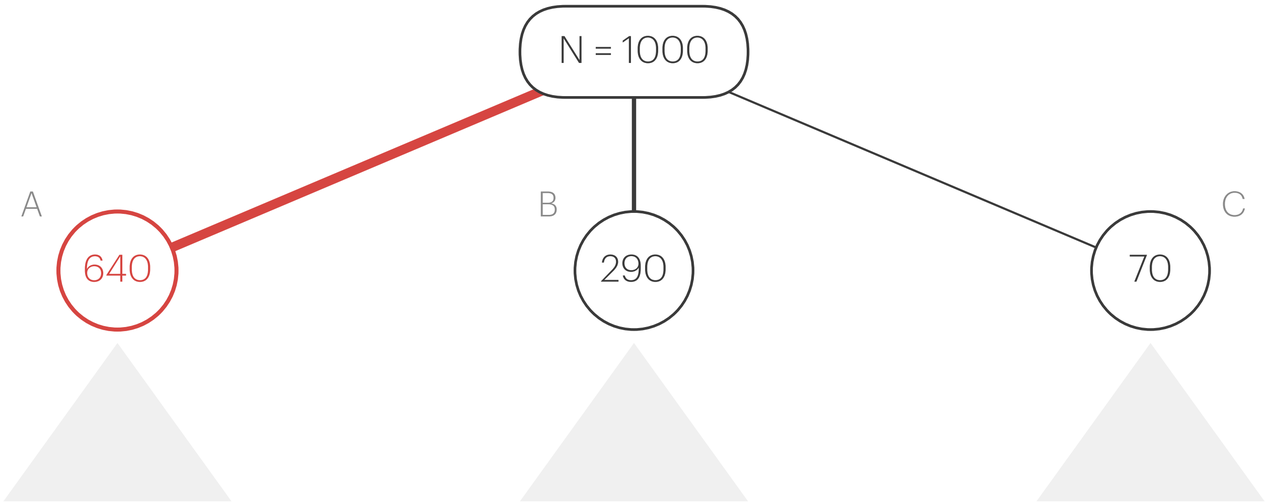
Simulate: random playout from C1, and we win

One Simulation, Step by Step



Backpropagate: each node counts wins for whoever moved into it

One Simulation, Step by Step



visits

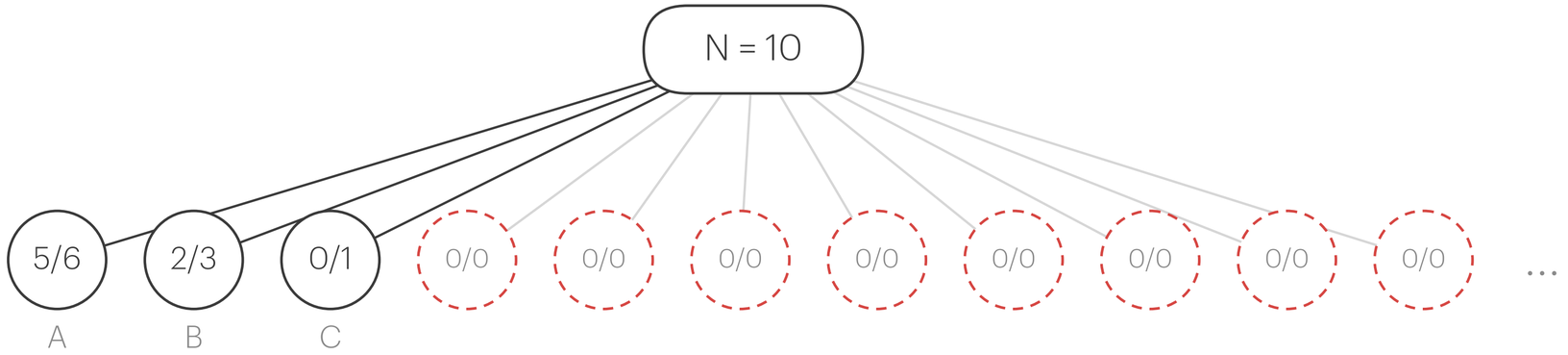
After many simulations: play the most-visited move

Visits, not win rate: robust to lucky streaks

Where Plain MCTS Gets Stuck

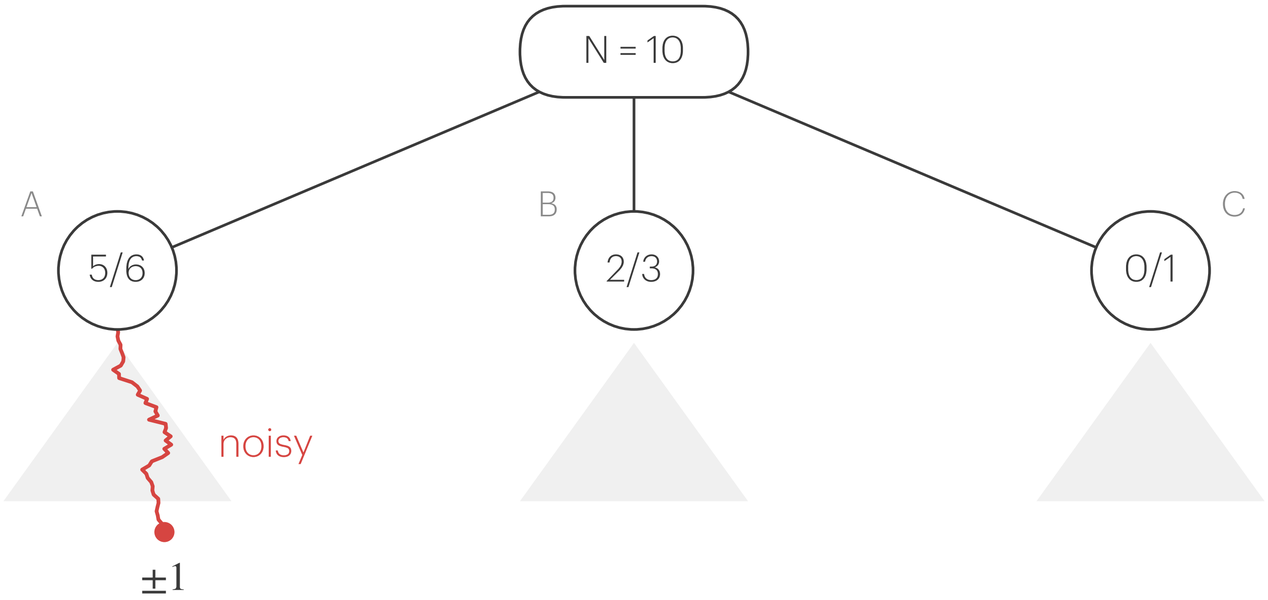
Every move must be tried once before UCT can compare them

An unvisited move scores infinity



Where Plain MCTS Gets Stuck

Random playouts judge positions badly



Where Plain MCTS Gets Stuck

Random playouts judge positions badly

Strong amateur level, but no further

Crazy Stone, Zen and others, 2006–2015

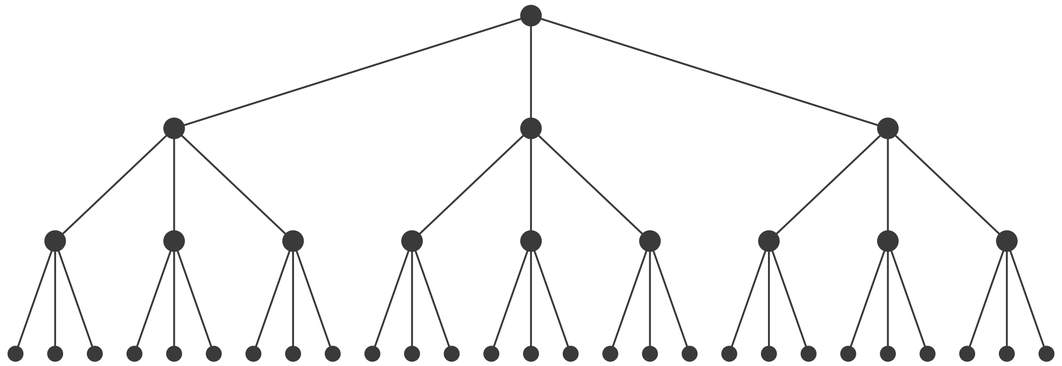
What if we had good intuition about moves and positions?

A policy and a value. Let's learn them!

Inside AlphaGo

Search, meet intuition

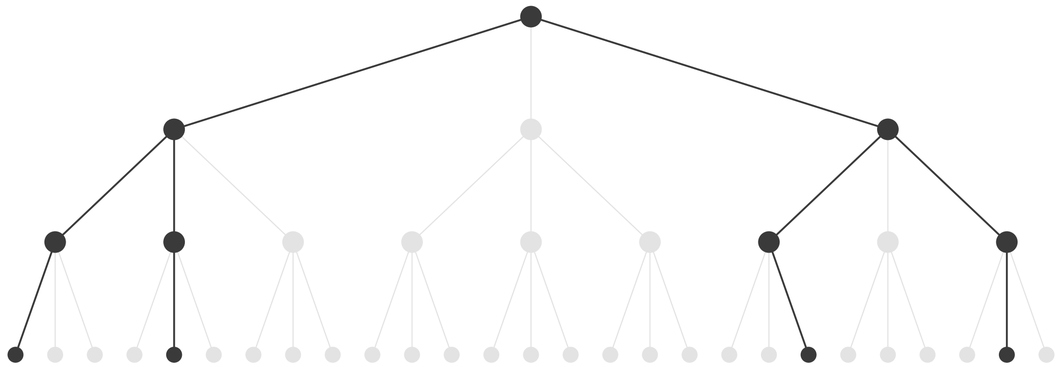
The Plan: MCTS, but Clever



We'll still run MCTS, but search cleverly

Plain MCTS tries every move and plays every game to the end

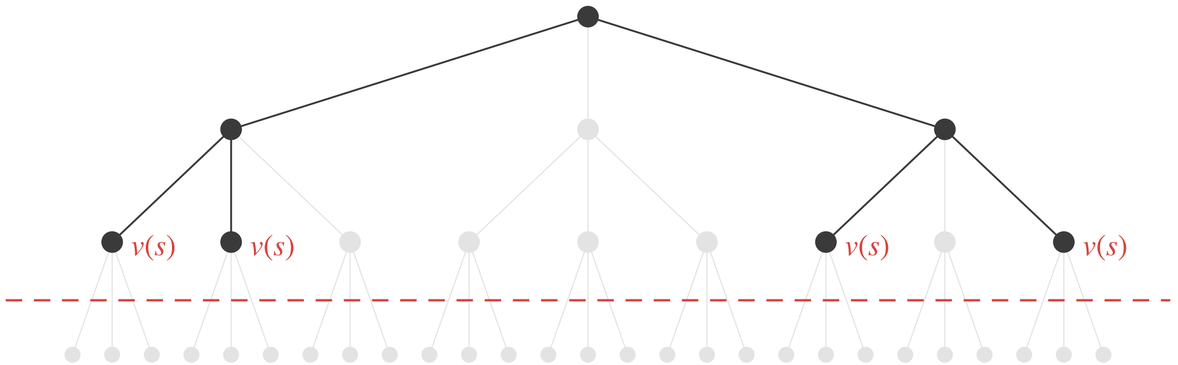
The Plan: MCTS, but Clever



Breadth: a policy network picks which moves to try

a policy $p(a \mid s)$

The Plan: MCTS, but Clever

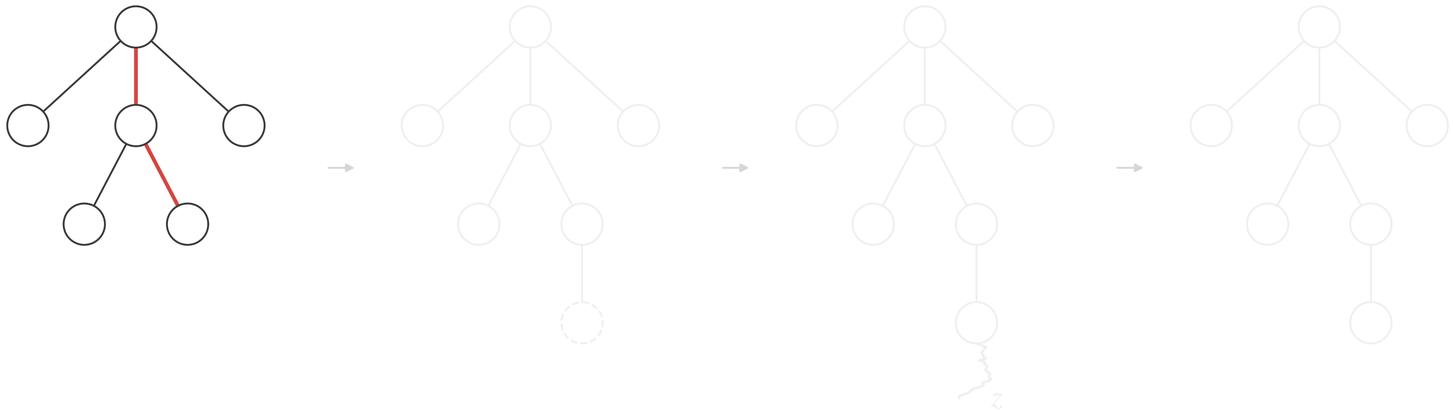


Depth: a value network says who's winning, so we stop early

a value $v(s)$

Change 1: UCT to PUCT

Selection picked children by UCT



Selection Expansion Simulation Backpropagation

$$\mathrm{UCT}(i) = \underbrace{\frac{W_i}{N_i}}_{\text{win rate}} + \underbrace{c\sqrt{\frac{\ln N}{N_i}}}_{\text{explore}}$$

Change 1: UCT to PUCT

$$\text{UCT} = Q(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}}$$

$$\text{PUCT} = Q(s,a) + c\,P(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)}$$

$P(s,a)$: how promising the policy network thinks move a is

Q is the average value so far, in $[-1, +1]$

Change 1: UCT to PUCT

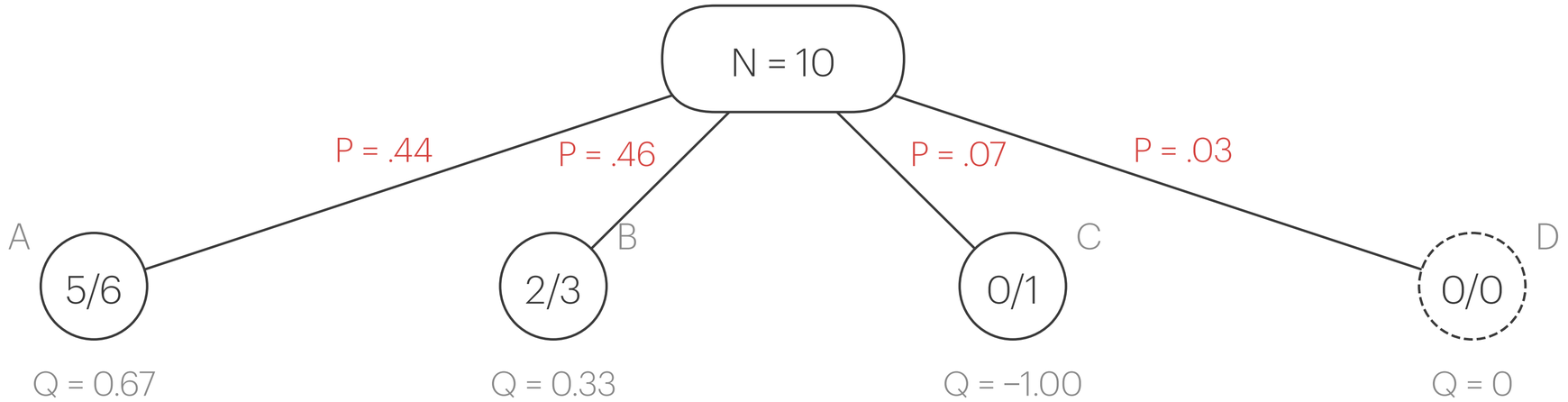
$$\text{UCT} = Q(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}}$$

$$\text{PUCT} = Q(s,a) + c\,P(s,a)\,\frac{\sqrt{N(s)}}{1+N(s,a)}$$

An untried move: UCT gives ∞, PUCT gives $c \cdot P \cdot \sqrt{N(s)}$

Promising moves first; bad moves late, not never

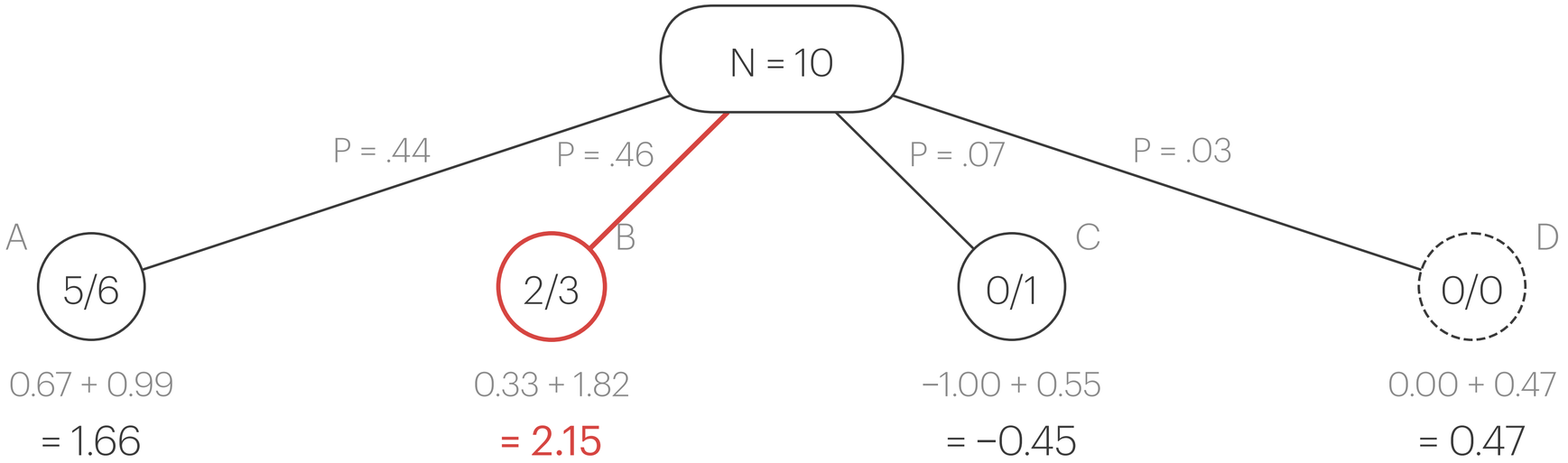
Which Child Does PUCT Pick?



Same tree, now with priors from p

$c = 5$, values in $[-1, +1]$

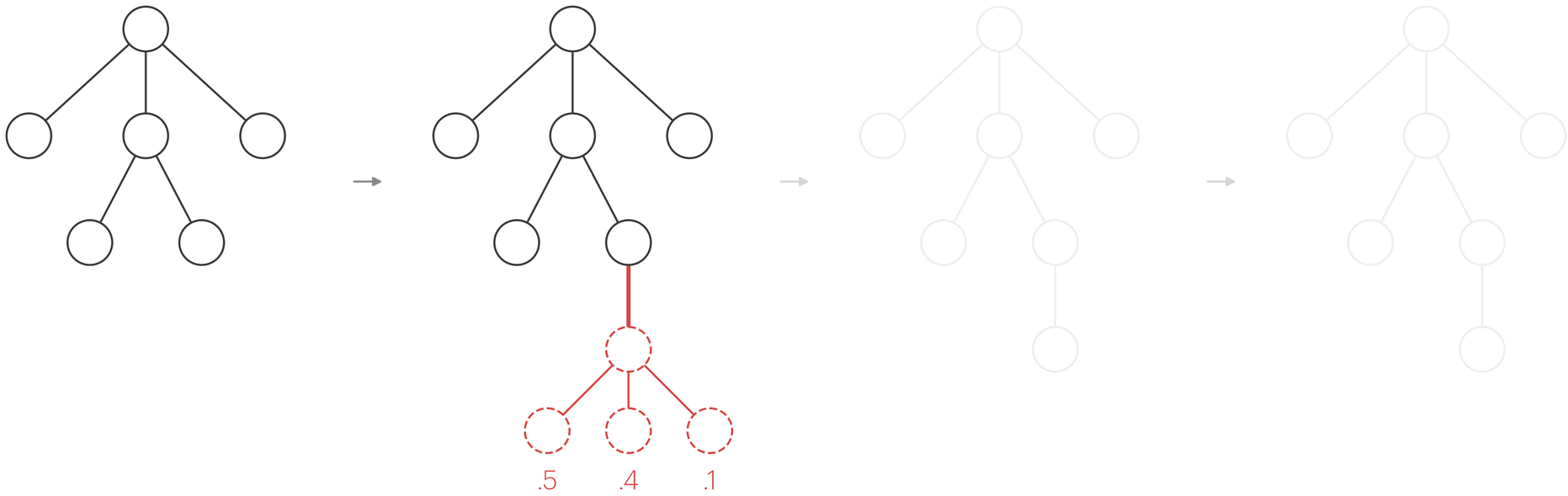
Which Child Does PUCT Pick?



UCT would try D (its score is ∞), then C. PUCT picks B.

Each score is Q plus a bonus that grows with the prior

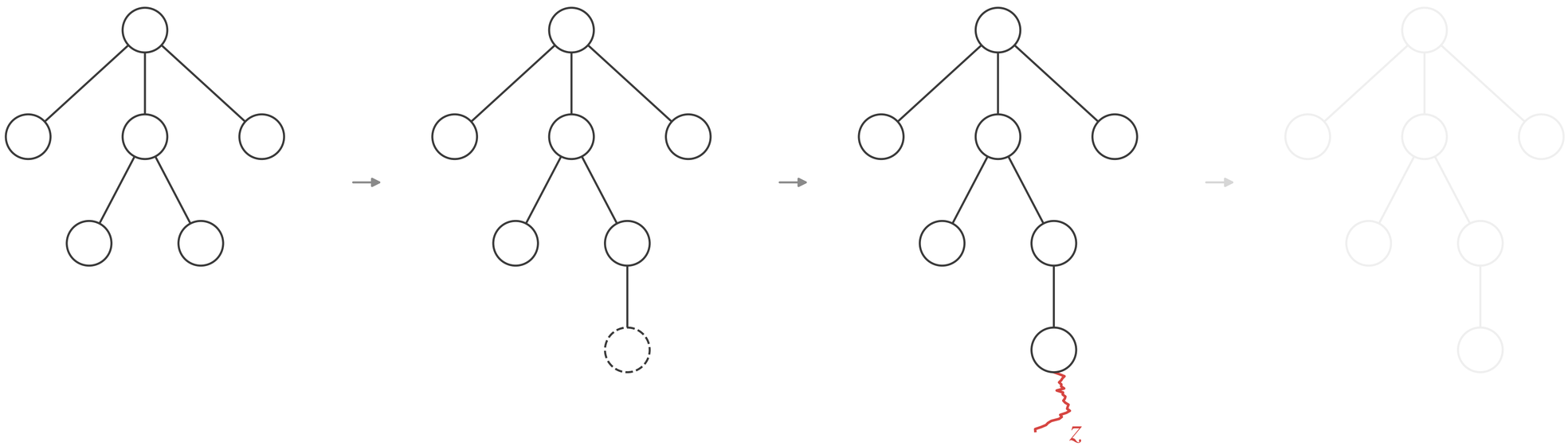
Which Child Does PUCT Pick?



At a new leaf, ask p once and initialise all its children

No more trying every move once first

Change 2: Don't Play It Out

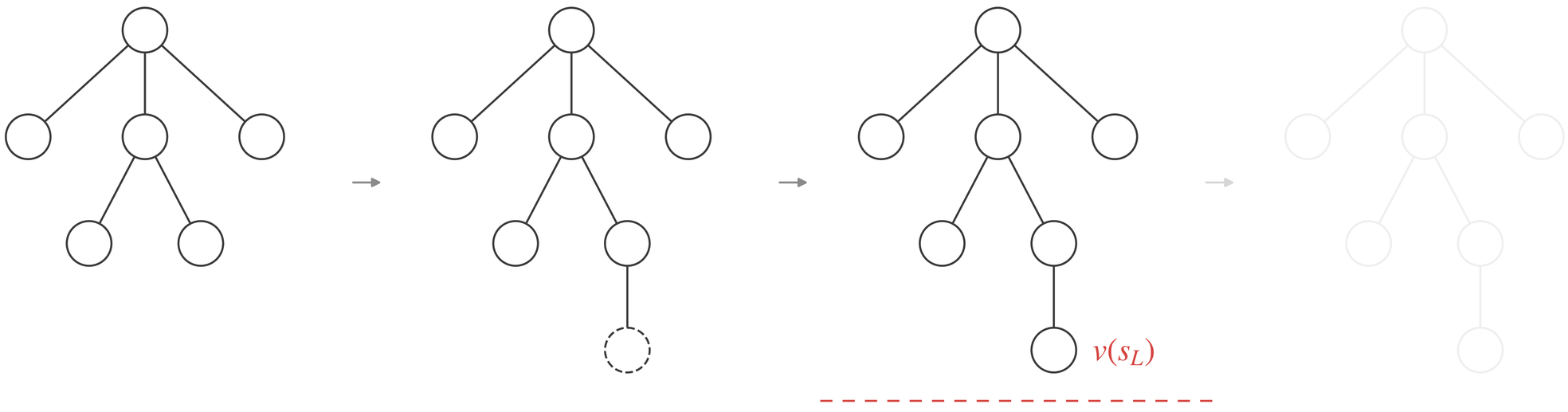


Selection Expansion Simulation Backpropagation

Simulation: play randomly until the game ends

Remember the plan: stop early, and ask who's winning

Change 2: Don't Play It Out



Chop it off: ask the value network for $v(s)$

One network evaluation of the leaf instead of a playout

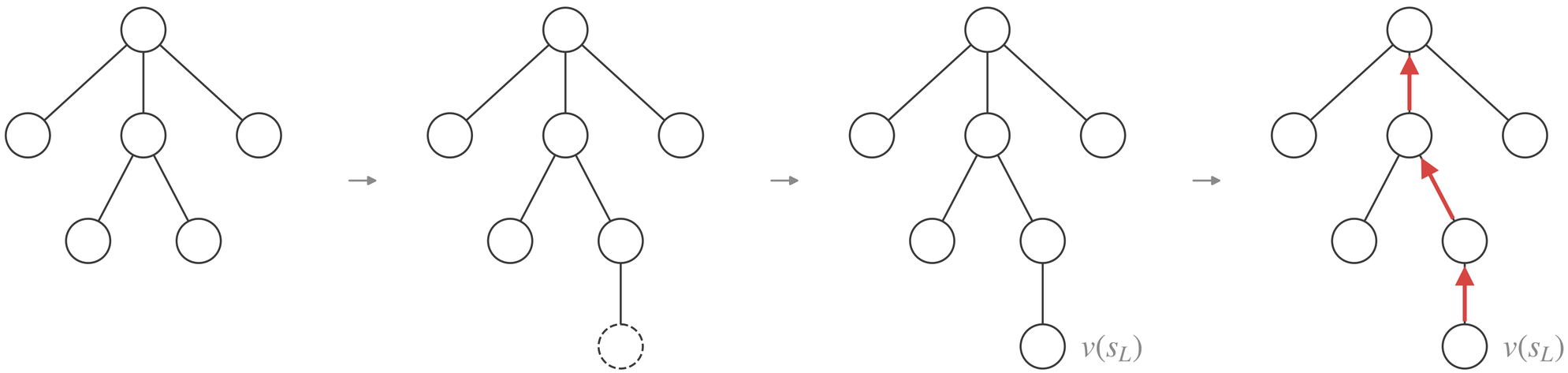
Change 2: Don't Play It Out

$$V(s_L) = (1 - \lambda) \underbrace{v_\theta(s_L)}_{\text{value net}} + \lambda \underbrace{z_L}_{\text{fast rollout}}$$

AlphaGo mixed both, with $\lambda = ½$

One value network call plus one fast rollout

Change 2: Don't Play It Out

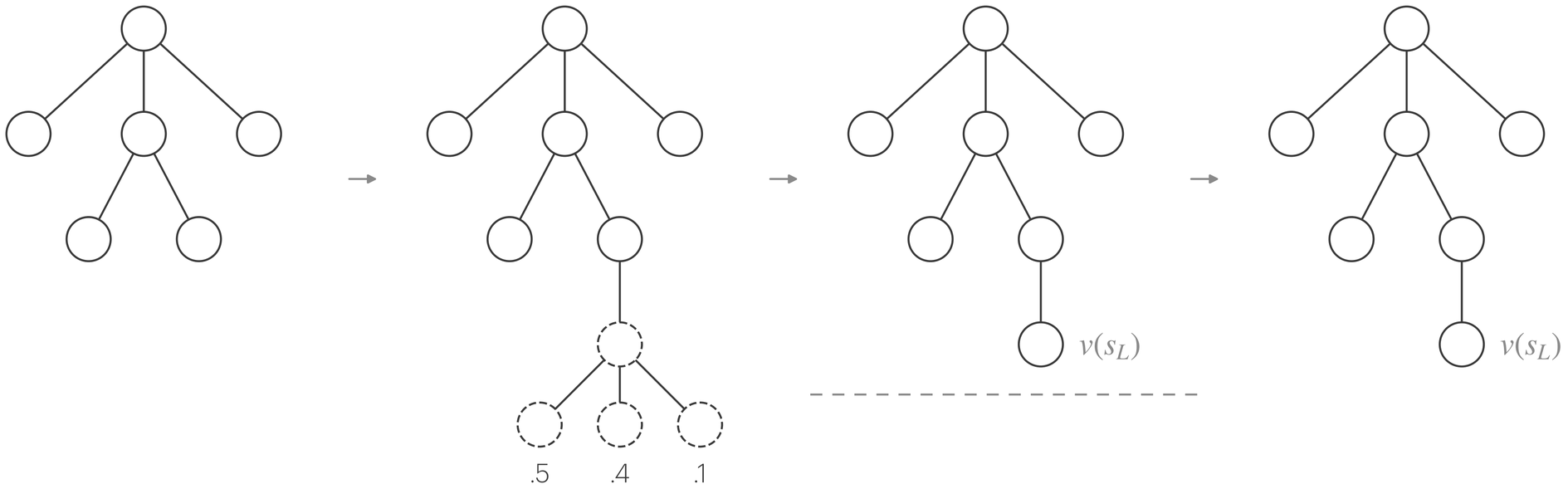


Selection Expansion Evaluation Backup

Backup: each node's Q is the mean of the values below it

Then, as before, play the most visited move

All We Need: p and v



Selection

p

Expansion

p

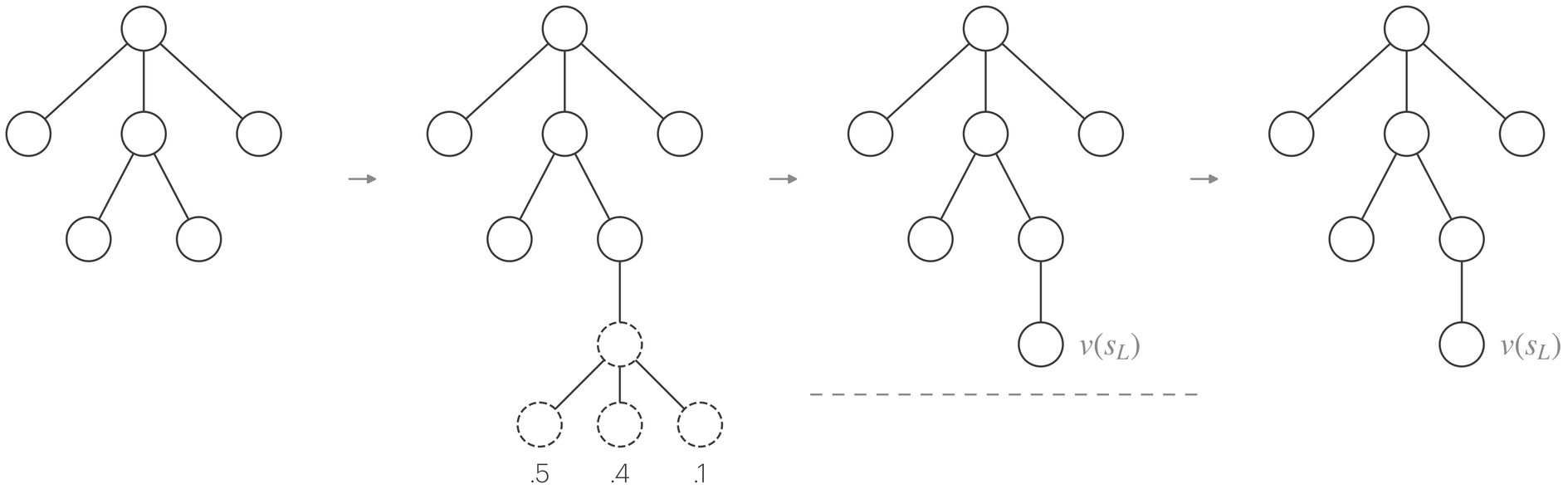
Evaluation

v

Backup

p steers Selection and Expansion, v replaces the playout

All We Need: p and v



Selection

p

Expansion

p

Evaluation

v

Backup

If we have p and v, we're good to go

Everything else is the MCTS we already know

Getting p and v



Imitate humans: predict the expert's next move

30M positions, 57% accurate. This p gives the priors $P(s,a)$

Who Do We Play Against?

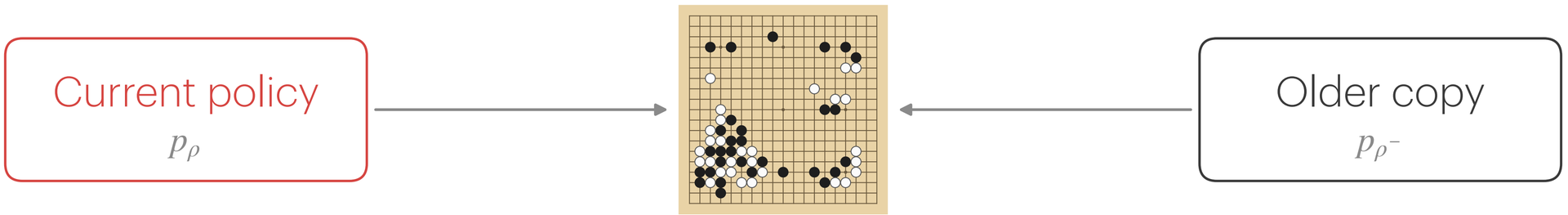


REINFORCE needs episodes

In a two-player game, that means playing against an opponent

Who Do We Play Against?





Play against a copy of yourself

Who Do We Play Against?

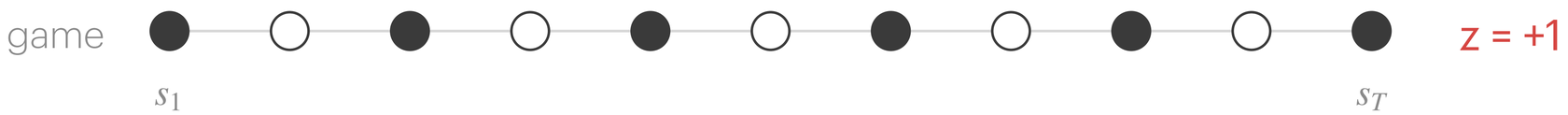


The opponent is always about as strong as you

No human data needed: an automatic curriculum

One Game, One Episode

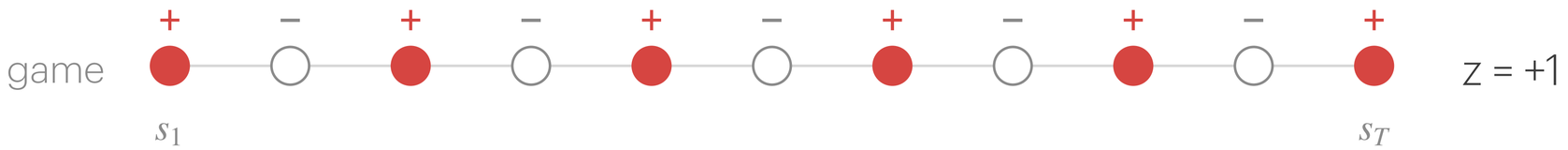




Reward only at the end: $z = +1$ if you win, -1 if you lose

One Game, One Episode





The winner's moves become more likely, the loser's less

One Game, One Episode



It's the REINFORCE update from before

$$\Delta\rho \propto \underbrace{\nabla_\rho \log p_\rho(a_t \mid s_t)}_{\text{score function}} \underbrace{z_t}_{\text{outcome}}$$

The opponent is simply part of the environment

Did Self-Play Help?





The RL policy beats the SL policy in over 80% of games

The SL policy it started from

Getting p and v

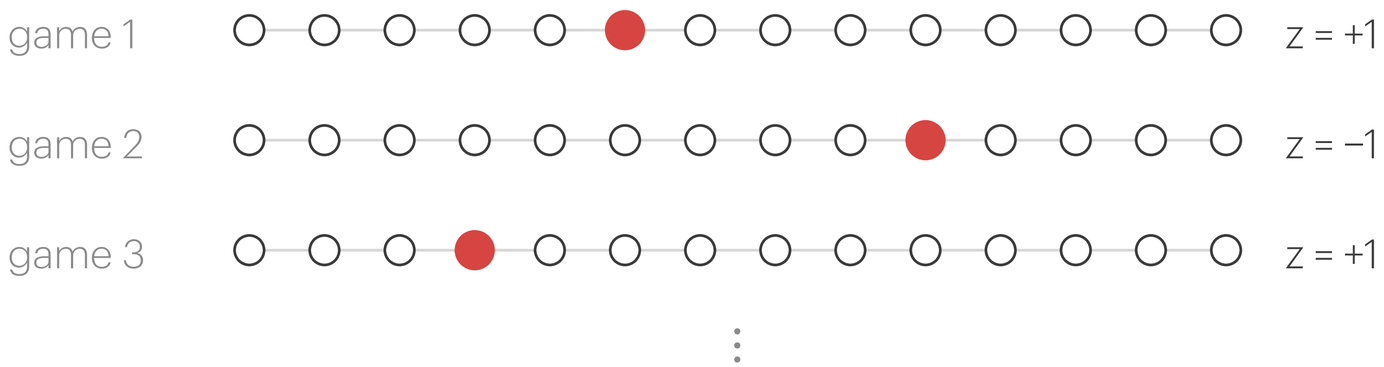


Now use the RL policy to learn the value function

$$v_\theta(s) \approx v^{p_\rho}(s) = \mathbb{E}\,[\,z \mid s\,]$$

Let the RL policy play itself, and regress each game's outcome z

Getting p and v



Use one position per game

Positions from one game look alike, so the net would memorise the outcome

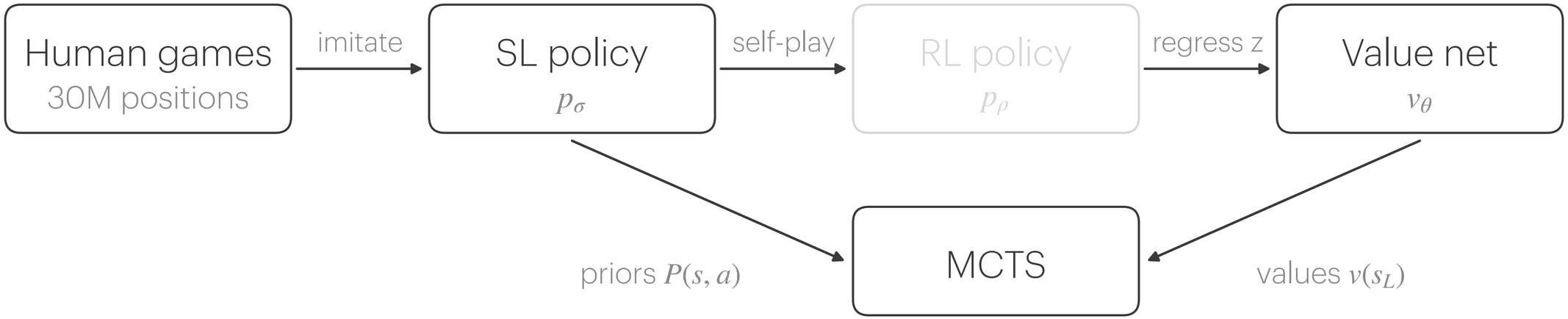
Getting p and v

Once we have v, use it as the baseline

$$\Delta\rho \propto \underbrace{\nabla_\rho \log p_\rho(a_t \mid s_t)}_{\text{score function}} \quad \underbrace{(z_t - v(s_t))}_{\text{outcome – baseline}}$$

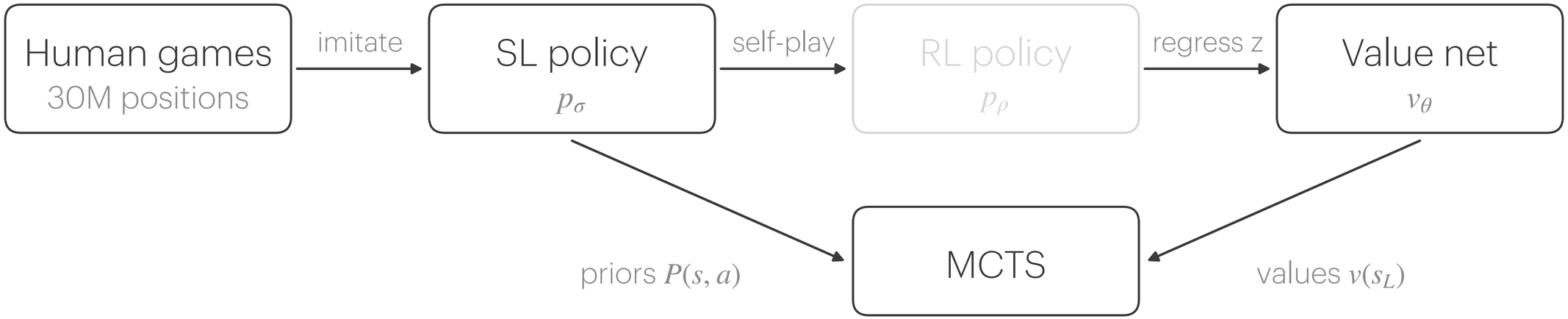
In the paper: baseline 0 on the first pass, v on the second: a small boost

Which Policy Does MCTS Use?



The RL policy only made the games for $\boldsymbol{v}$

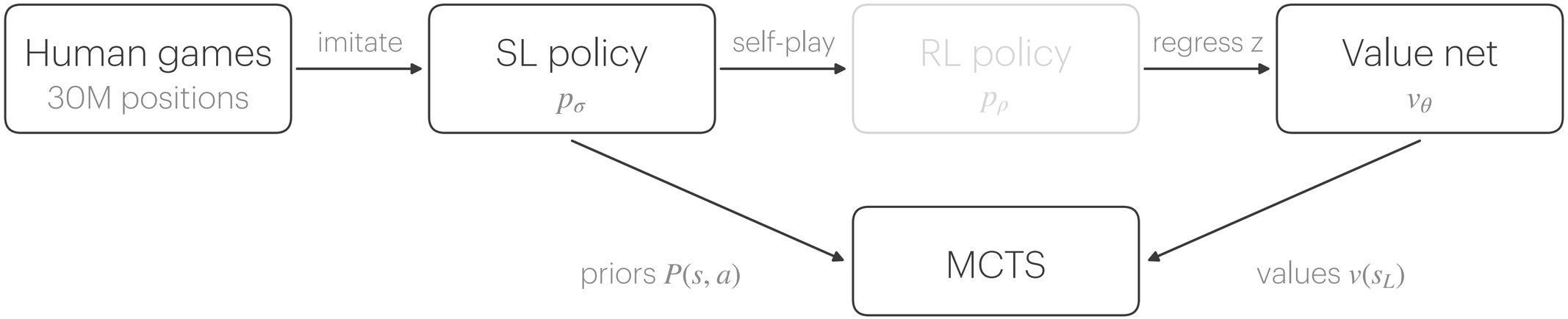
Which Policy Does MCTS Use?



Crucially, it is not the policy used during MCTS

The search uses the SL policy for its priors

Which Policy Does MCTS Use?



Why? Humans suggest a diverse set of good moves

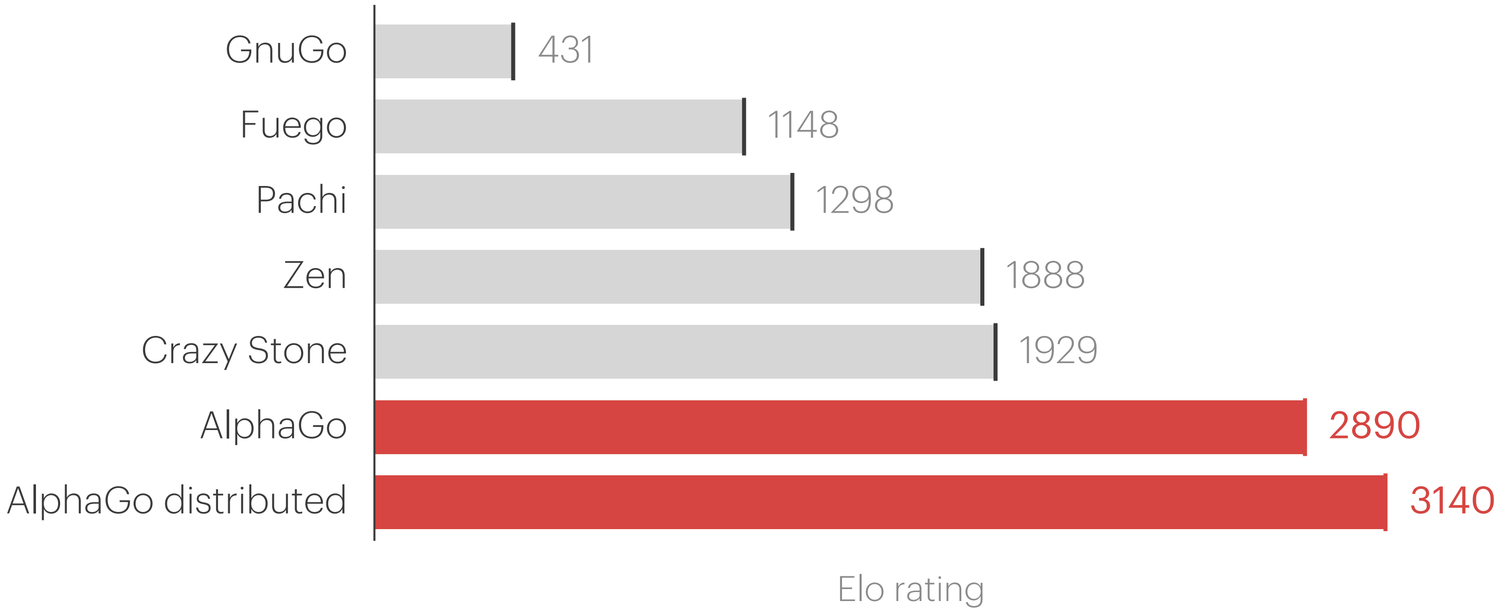
RL bets on the single best one

But v from the stronger RL policy is the better value

How Strong Was It?

Won 494 of 495 games against the best Go programs (99.8%)

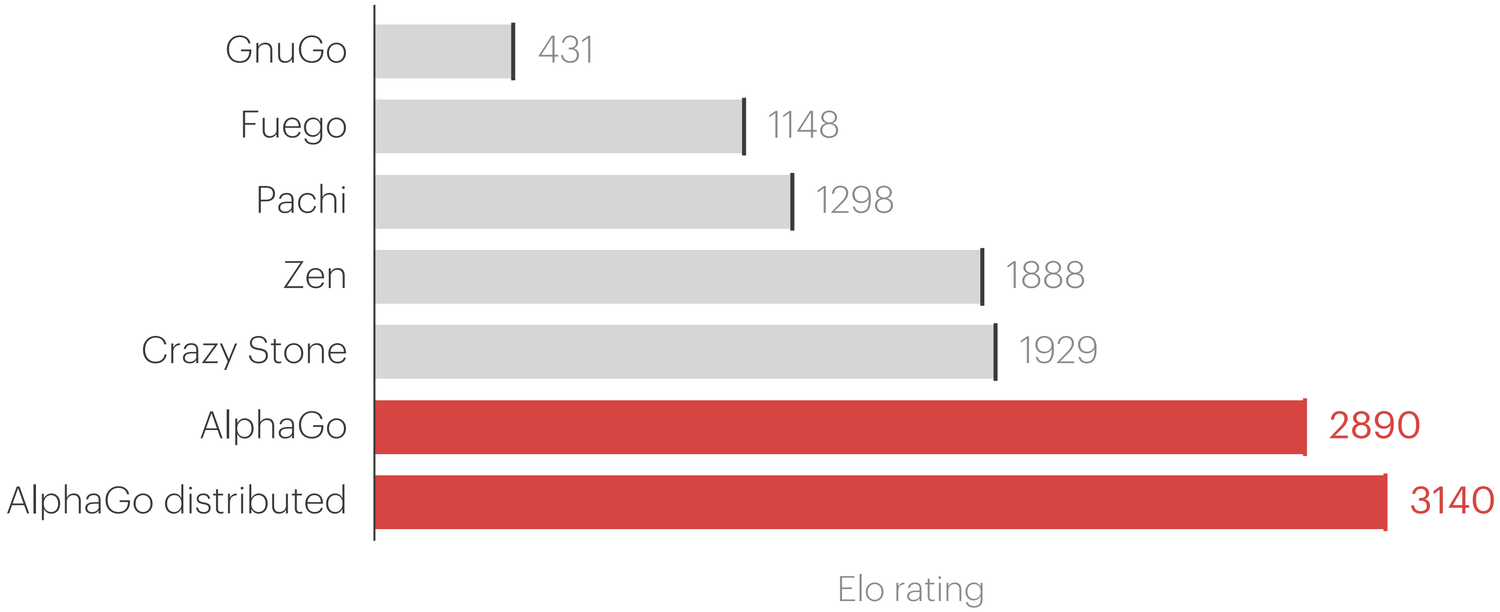
Elo: 230 points ahead ≈ 79% chance to win



How Strong Was It?

Even with 4 free moves at the start, they lost

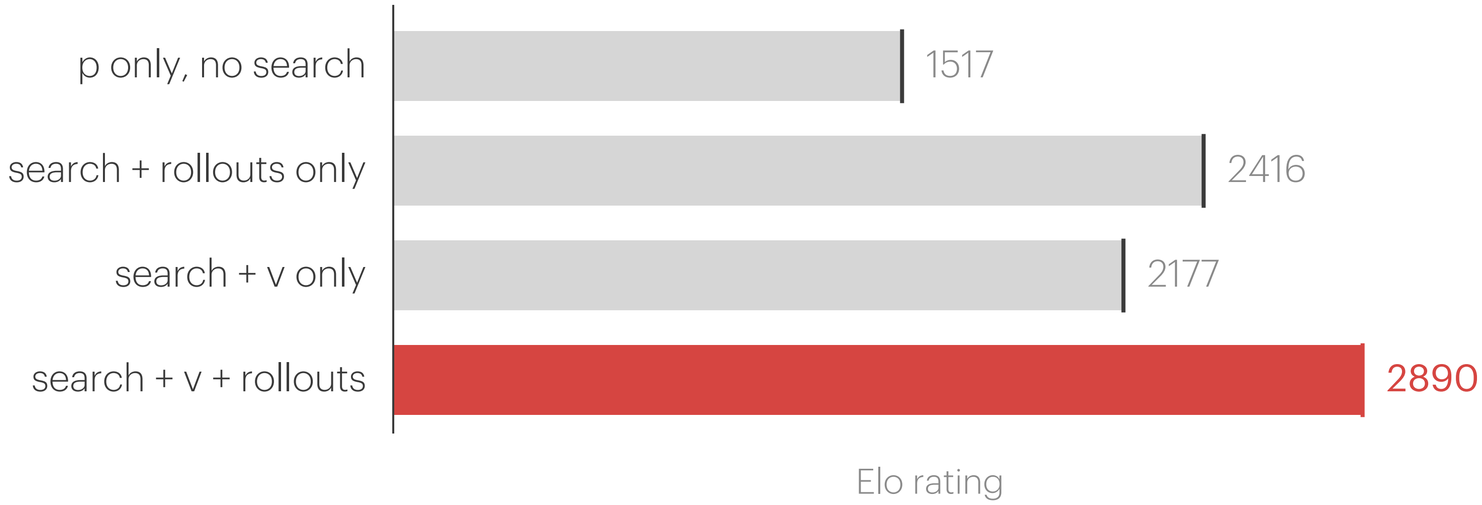
AlphaGo won 77%, 86% and 99% of handicap games vs Crazy Stone, Zen and Pachi



Do We Need Both p and v?

Every piece helps; together they are best

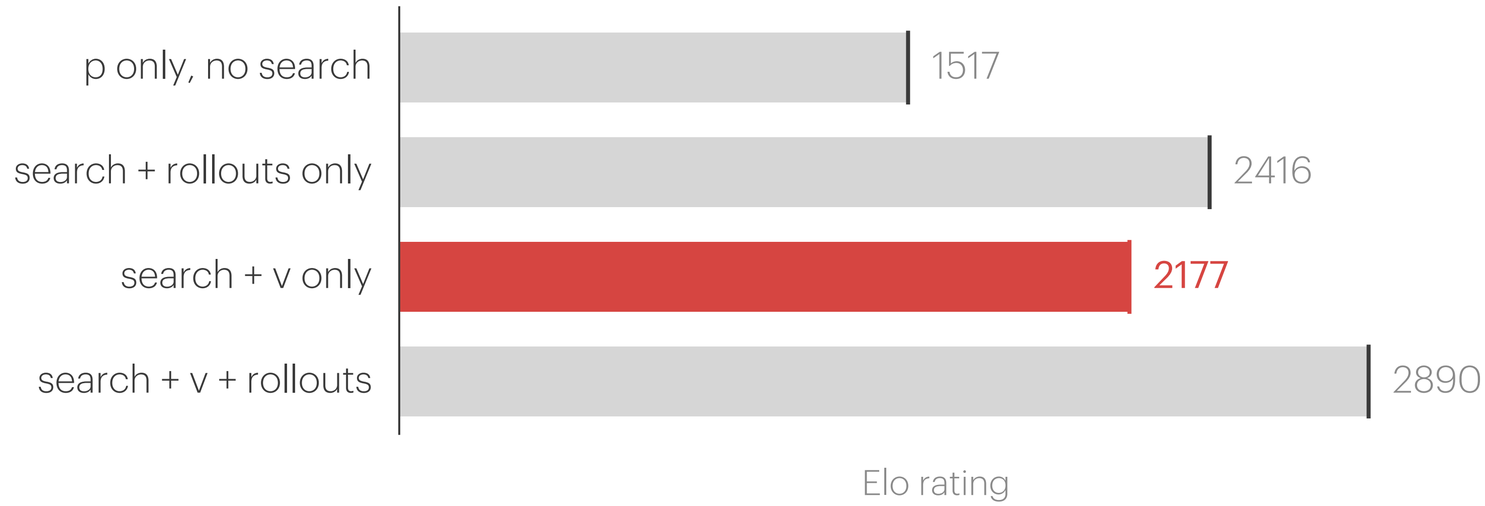
Strongest: mix v and the fast rollout, $\lambda = ½$



Do We Need Both p and v?

v alone already beats every other Go program

2177 vs 1929 for Crazy Stone: a learned value can replace the playout



Beating a Professional

October 2015: AlphaGo beat Fan Hui 5–0

European champion 2013–2015, professional 2 dan; distributed AlphaGo

First program to beat a professional at full-size Go, no handicap

Thought to be at least a decade away